```
==================== backend/main.py ====================
import os
os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
for _var in ("ALL_PROXY", "all_proxy", "HTTPS_PROXY", "https_proxy",
             "HTTP_PROXY", "http_proxy"):
    os.environ.pop(_var, None)
from contextlib import asynccontextmanager
from pathlib import Path
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse
from backend.api.chat import router as chat_router
from backend.api.knowledge import router as kb_router
from backend.api.analytics import router as analytics_router
from backend.api.multimodal import router as mm_router
from backend.api.digital_human import router as dh_router
from backend.api.asr import router as asr_router
from backend.api.trip import router as trip_router
from backend.api.tour import router as tour_router
from backend.api.theme import router as theme_router
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 初始化 SQLite（建表，幂等）
    from backend.db.database import init_db
    init_db()
    from backend.core.config import settings
    if settings.deploy_mode == "local":
        print("预热嵌入模型（BGE-M3）...")
        from backend.rag.embedder import get_embedder
        get_embedder()
        print("预热重排模型（BGE-Reranker）...")
        from backend.rag.retriever import get_reranker
        get_reranker()
        print("本地模型预热完成，服务就绪")
    else:
        print("云端模式（DashScope），服务就绪")
    # 后台预热客流统计：读 14 万行 xlsx 约 10~20s（openpyxl 慢），
    # 不预热的话首个 navigation/recommend 请求会被这次冷加载冻住整整 20s。
    # 放后台线程，不阻塞启动，等首个用户提问时缓存已热。
    import threading
    def _prewarm_crowd():
        try:
            from backend.analytics.crowd import _stats
            _stats()
            print("客流统计预热完成")
        except Exception as e:  # 缺 xlsx/pandas 不致命，导游不带客流上下文即可
            print(f"客流统计预热跳过: {type(e).__name__}: {e}")
    threading.Thread(target=_prewarm_crowd, daemon=True).start()
    yield
app = FastAPI(title="景区导览AI数字人", version="0.1.0", lifespan=lifespan)
```

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
app.include_router(chat_router)
app.include_router(mm_router)
app.include_router(kb_router)
app.include_router(analytics_router)
app.include_router(dh_router)
app.include_router(asr_router)
app.include_router(trip_router)
app.include_router(tour_router)
app.include_router(theme_router)
@app.get("/health")
def health():
    return {"status": "ok"}
@app.get("/test", include_in_schema=False)
def test_ui():
    """后端托管测试面板（PC 版），避免 file:// 跨域问题。"""
    p = Path(__file__).resolve().parent.parent / "test_ui.html"
    return FileResponse(p, media_type="text/html")
@app.get("/m", include_in_schema=False)
def mobile_ui():
    """移动优先的后端功能测试台（手机浏览器访问 /m）。"""
    p = Path(__file__).resolve().parent.parent / "test_mobile.html"
    return FileResponse(p, media_type="text/html")
@app.get("/talk", include_in_schema=False)
def talk_ui():
    """数字人交互页（WebRTC 数字人 + 问答走桥接 → 端到端语音问答演示）。"""
    p = Path(__file__).resolve().parent.parent / "talk.html"
    return FileResponse(p, media_type="text/html")
@app.get("/app", include_in_schema=False)
def guest_app():
    """游客端正式 UI（东方禅意主题）：全屏数字人 + 多模态(语音/拍照/文字) +
    主动迎宾 + 同行人格切换 + 个性化推荐 + 行程游记。赛题「游客交互端」交付页。"""
    p = Path(__file__).resolve().parent.parent / "app.html"
    return FileResponse(p, media_type="text/html")
@app.get("/dashboard", include_in_schema=False)
def dashboard_ui():
    """运营数据大屏（ECharts 可视化 analytics + 实时客流热度）。"""
    p = Path(__file__).resolve().parent.parent / "dashboard.html"
    return FileResponse(p, media_type="text/html")
@app.get("/echarts.min.js", include_in_schema=False)
def echarts_js():
    """本地服 ECharts（绕开浏览器代理对 CDN 的拦截）。"""
    p = Path(__file__).resolve().parent.parent / "echarts.min.js"
    return FileResponse(p, media_type="application/javascript")
==================== backend/rag/retriever.py ====================
```

```
"""
EA-RAG 检索器（Entity-Anchored Adaptive RAG，自研编排层算法）。
在「稠密(BGE-M3) + 稀疏(BM25) → 重排(BGE-Reranker)」基线之上，叠加三个自研机制
（均可经 settings.ea_* 独立开关，供消融实验）：
 ① 实体锚定 Entity Anchoring
    景区知识库的景点名表是已知先验（结构化 chunk 含"景点名称: X"）。
    query 与景点名做最长公共子串匹配，命中则该景点的 chunk 在融合分上加锚定加成。
    解决真实漏检：游客说"光明顶"，稠密检索却召回其他景点的 chunk。
 ② 查询特征自适应融合 Adaptive Fusion
    基线是 dense/sparse 候选简单拼接（隐式等权）。改为加权 RRF（倒数排名融合），
    权重按 query 特征动态调整：含数字/量词特征 → BM25 权重↑（精确匹配强）；
    口语化长句 → dense 权重↑（语义理解强）。
 ③ 置信度门控 Confidence Gating
    Reranker top1 分数（normalize 后 0~1）本身就是"知识库里有没有答案"的置信度，
    基线白白丢弃。低于阈值 → 返回空召回，触发兜底话术——把"拒答"从纯靠
    prompt 自觉，变成有数学依据的门控。
延迟优化（沿袭基线）：
- 云端模式 retrieve_async() 用 asyncio.gather 让 DashScope embed 和 BM25 并行
"""
import asyncio
import os
import re
from difflib import SequenceMatcher
from typing import List
os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
for _var in ("ALL_PROXY", "all_proxy", "HTTPS_PROXY", "https_proxy",
             "HTTP_PROXY", "http_proxy"):
    os.environ.pop(_var, None)
import chromadb
from rank_bm25 import BM25Okapi
from backend.core.config import settings
_reranker = None
# RRF 平滑常数（业界标准取 60）；锚定加成约为 RRF rank1 分值的 1.2 倍，
# 足以把命中景点的 chunk 顶进前排，又不至于完全压过相关性排序
_RRF_K = 60
_ANCHOR_BONUS = 0.02
def get_reranker():
    global _reranker
    if _reranker is None:
        from huggingface_hub import snapshot_download
        from FlagEmbedding import FlagReranker
        local_path = snapshot_download(
            repo_id=settings.reranker_model,
            ignore_patterns=["*.DS_Store", "imgs/*", "*.h5", "flax_model*",
                             "rust_model*", "tf_model*"],
        )
        _reranker = FlagReranker(local_path, use_fp16=True)
    return _reranker
def get_collection(scenic_id: str) -> chromadb.Collection:
    client = chromadb.PersistentClient(path=str(settings.vector_store_dir))
```

```
    return client.get_or_create_collection(
        name=f"scenic_{scenic_id}",
        metadata={"hnsw:space": "cosine"},
    )
def _embed_query_sync(query: str) -> List[float]:
    if settings.deploy_mode == "cloud":
        from backend.rag.embedder_dashscope import embed_texts_cloud
        return embed_texts_cloud([query])[0]
    from backend.rag.embedder import embed_texts
    return embed_texts([query])[0]
def _bm25_search(query: str, all_docs: List[str], top_k: int) -> List[str]:
    tokenized = [list(d) for d in all_docs]
    bm25 = BM25Okapi(tokenized)
    scores = bm25.get_scores(list(query))
    top_k = min(top_k, len(all_docs))
    indices = sorted(range(len(scores)), key=lambda i: scores[i], reverse=True)[:top_k]
    return [all_docs[i] for i in indices]
def _dense_search(query_vec: List[float], collection: chromadb.Collection,
                  dense_k: int) -> List[str]:
    result = collection.query(
        query_embeddings=[query_vec],
        n_results=dense_k,
        include=["documents"],
    )
    return result["documents"][0]
# ═══════════════════════════════════════════════════════════════
# ① 实体锚定
# ═══════════════════════════════════════════════════════════════
_entity_cache: dict[str, list[str]] = {}
_ENTITY_PATTERN = re.compile(r"景点名称[:：]\s*([^|\n]+)")
def _get_entities(scenic_id: str, all_docs: List[str]) -> list[str]:
    """从结构化 chunk 中提取该景区全部景点名（每景区缓存一次）。"""
    if scenic_id not in _entity_cache:
        names = []
        for d in all_docs:
            m = _ENTITY_PATTERN.search(d)
            if m:
                name = m.group(1).strip()
                if name and name not in names:
                    names.append(name)
        _entity_cache[scenic_id] = names
    return _entity_cache[scenic_id]
def invalidate_entity_cache(scenic_id: str | None = None) -> None:
    """知识库更新后调用，清空景点名缓存。"""
    if scenic_id is None:
        _entity_cache.clear()
    else:
        _entity_cache.pop(scenic_id, None)
def _match_entities(query: str, entities: list[str]) -> list[str]:
    """
```

```
    query 与景点名匹配。匹配条件（满足其一）：
     - 景点名完整出现在 query 中；
     - 最长公共子串 ≥ max(2, 景点名长度的一半)——容忍口语省略
       （"大照壁多长" 命中 "灵山大照壁"）。
    """
    matched = []
    for ent in entities:
        if ent in query:
            matched.append(ent)
            continue
        m = SequenceMatcher(None, query, ent).find_longest_match(
            0, len(query), 0, len(ent))
        if m.size >= max(2, len(ent) // 2):
            matched.append(ent)
    return matched
# ═══════════════════════════════════════════════════════════════
# ② 查询特征自适应融合（加权 RRF）
# ═══════════════════════════════════════════════════════════════
# 数字/量词类问题特征：精确匹配（BM25）更可靠
_FACTUAL_PATTERN = re.compile(
    r"\d|几|多少|哪一?年|多高|多大|多长|多宽|多深|多重|面积|高度|长度|票价|价格")
def _fusion_weights(query: str) -> tuple[float, float]:
    """返回 (w_dense, w_sparse)。默认等权；按 query 特征自适应。"""
    if not settings.ea_adaptive_fusion:
        return 1.0, 1.0
    w_dense, w_sparse = 1.0, 1.0
    if _FACTUAL_PATTERN.search(query):
        w_sparse = 1.4        # 数字/量词 → 关键词精确匹配更重要
    elif len(query) >= 18:
        w_dense = 1.3         # 口语化长句 → 语义理解更重要
    return w_dense, w_sparse
def _fuse(query: str, dense_docs: List[str], sparse_docs: List[str],
          all_docs: List[str], scenic_id: str) -> List[str]:
    """
    加权 RRF 融合 + 实体锚定加成，返回按融合分排序的候选。
    关闭两机制时退化为基线行为（等权 RRF ≈ 原拼接去重的排序版）。
    """
    w_dense, w_sparse = _fusion_weights(query)
    scores: dict[str, float] = {}
    for rank, doc in enumerate(dense_docs):
        scores[doc] = scores.get(doc, 0.0) + w_dense / (_RRF_K + rank + 1)
    for rank, doc in enumerate(sparse_docs):
        scores[doc] = scores.get(doc, 0.0) + w_sparse / (_RRF_K + rank + 1)
    if settings.ea_entity_anchor:
        matched = _match_entities(query, _get_entities(scenic_id, all_docs))
        if matched:
            # 命中景点的 chunk 加锚定加成；不在候选里的也补进来（兜住漏检）
            for doc in all_docs:
                if any(ent in doc for ent in matched):
                    scores[doc] = scores.get(doc, 0.0) + _ANCHOR_BONUS
```

```
    return [d for d, _ in sorted(scores.items(), key=lambda x: x[1], reverse=True)]
# ═══════════════════════════════════════════════════════════════════════
# ③ 置信度门控 + 主检索入口
# ═══════════════════════════════════════════════════════════════════════
def retrieve(query: str, scenic_id: str) -> List[str]:
    """同步检索（本地模式 + 兜底）。"""
    collection = get_collection(scenic_id)
    total_docs = collection.count()
    if total_docs == 0:
        return []
    query_vec = _embed_query_sync(query)
    dense_k = min(settings.dense_top_k, total_docs)
    dense_docs = _dense_search(query_vec, collection, dense_k)
    all_docs: List[str] = collection.get(include=["documents"])["documents"]
    sparse_docs = _bm25_search(query, all_docs, settings.sparse_top_k)
    candidates = _fuse(query, dense_docs, sparse_docs, all_docs, scenic_id)
    if settings.deploy_mode == "cloud":
        return candidates[: settings.rerank_top_k]
    reranker = get_reranker()
    # 候选可能因锚定补漏而变多，重排只精算前 16 条控制延迟
    candidates = candidates[:16]
    pairs = [[query, doc] for doc in candidates]
    rerank_scores = reranker.compute_score(pairs, normalize=True)
    if not isinstance(rerank_scores, list):
        rerank_scores = [rerank_scores]
    ranked = sorted(zip(rerank_scores, candidates), key=lambda x: x[0], reverse=True)
    # 置信度门控：连最相关的候选都低于阈值 → 知识库无此信息，触发兜底
    if settings.ea_confidence_gate and ranked and ranked[0][0] < settings.ea_gate_threshold:
        return []
    return [doc for _, doc in ranked[: settings.rerank_top_k]]
async def retrieve_async(query: str, scenic_id: str) -> List[str]:
    """
    异步检索（云端模式专用）。
    关键优化：DashScope embed（~400ms 网络IO）和 BM25（~30ms CPU）并行执行。
    云端无 reranker，门控不生效（融合与锚定生效）。
    """
    collection = get_collection(scenic_id)
    total_docs = collection.count()
    if total_docs == 0:
        return []
    # 提前取所有文档（BM25 需要），这一步很快（本地 SQLite）
    all_docs: List[str] = collection.get(include=["documents"])["documents"]
    loop = asyncio.get_event_loop()
    # 并行：embed（网络IO）+ BM25（CPU）
    query_vec, sparse_docs = await asyncio.gather(
        loop.run_in_executor(None, _embed_query_sync, query),
        loop.run_in_executor(None, _bm25_search, query, all_docs, settings.sparse_top_k),
    )
    dense_k = min(settings.dense_top_k, total_docs)
    dense_docs = _dense_search(query_vec, collection, dense_k)
```

```
    candidates = _fuse(query, dense_docs, sparse_docs, all_docs, scenic_id)
    return candidates[: settings.rerank_top_k]
==================== backend/rag/pipeline.py ====================
"""
RAG 完整流水线：query → 检索 → LLM → 回复 + 情绪 → 存库。
延迟优化层次：
  1. LRU 缓存命中 → 直接返回，<5ms
  2. 异步并行检索 → embed 与 BM25 同时跑，省 ~50ms
  3. fast=True   → 用 qwen-turbo，首 token ~150ms vs max ~600ms
  4. 流式输出    → 第一句话完成即触发 TTS，不等全文
"""
import asyncio
from backend.rag.retriever import retrieve, retrieve_async
from backend.rag.cache import answer_cache
from backend.llm.prompts import build_system_prompt, QUERY_REWRITE_PROMPT, needs_rewrite
from backend.llm.client import chat, chat_stream, extract_emotion
from backend.db.database import save_conversation
from backend.core.config import settings
from backend.agent.orchestrator import classify
from backend.agent.handlers import intent_directive, skip_retrieval
from backend.agent.intents import INTENT_LABELS, Intent
from backend.analytics.crowd import crowd_brief
from backend.analytics.env_context import env_brief
# 路线/推荐意图才注入实时客流 + 时间天气，让导游给「错峰/避暑/带伞」等随机应变建议
_CROWD_INTENTS = {Intent.NAVIGATION, Intent.RECOMMEND}
# 按意图选模型：qa（事实问答）守 qwen-max 保 97.7% 准确率；其余生成性意图
# （路线/推荐/安抚/闲聊）切 qwen-turbo——事实已由注入的 RAG context 约束，生成质量足够，
# 而整段 LLM 时延实测 28s→4s（16→69 字/s）。这些意图不在准确率测试集范畴，不影响硬指标。
_FAST_INTENTS = {Intent.NAVIGATION, Intent.RECOMMEND, Intent.EMPATHY, Intent.CHITCHAT}
def _directive_with_context(intent: Intent, scenic_id: str) -> str:
    """意图指令 +（路线/推荐时）实时客流 + 时间天气环境上下文。"""
    directive = intent_directive(intent)
    if intent in _CROWD_INTENTS:
        for brief in (crowd_brief(scenic_id), env_brief(scenic_id)):
            if brief:
                directive += "\n\n" + brief
    return directive
SCENIC_CONFIG: dict[str, dict] = {
    "lingshan":   {"name": "灵山胜境",        "guide_name": "慧昭"},
    "nihuawan":   {"name": "拈花湾",          "guide_name": "禅悦"},
    "xihu":       {"name": "西湖",            "guide_name": "婉清"},
    "huangshan":  {"name": "黄山",            "guide_name": "云松"},
    "gugong":     {"name": "故宫博物院",      "guide_name": "瑾瑜"},
    "zhangjiajie": {"name": "张家界国家森林公园", "guide_name": "岚溪"},
    "bingmayong": {"name": "秦始皇帝陵博物院", "guide_name": "秦韵"},
    "mogaoku":    {"name": "敦煌莫高窟",      "guide_name": "妙音"},
}
def get_scenic_config(scenic_id: str) -> dict:
    return SCENIC_CONFIG.get(scenic_id, {"name": scenic_id, "guide_name": "小导"})
def rewrite_query(query: str, history: list[dict]) -> str:
```

```
    """
    用 qwen-turbo 把口语/含指代的问题改写为检索友好形式。
    只在必要时触发（含指代词或历史有上下文），避免浪费 API。
    """
    if not needs_rewrite(query, history):
        return query
    history_text = "\n".join(
        f"{'游客' if m['role']=='user' else '导游'}：{m['content'][:60]}"
        for m in history[-4:]  # 只看最近两轮
    ) or "（无历史）"
    prompt = QUERY_REWRITE_PROMPT.format(history=history_text, query=query)
    rewritten = chat(prompt, [], "", fast=True).strip()
    # 安全回退：改写失败或过长时用原始 query
    return rewritten if rewritten and len(rewritten) < 100 else query
def answer(
    query: str,
    scenic_id: str,
    history: list[dict] | None = None,
    session_id: str = "anonymous",
    fast: bool = False,
    lang: str = "auto",
    group_type: str = "adult",
) -> dict:
    """
    同步问答。fast=True 使用 qwen-turbo（延迟减半，适合简单问答）。
    lang: "auto"（跟随提问语言）或语言代码（en/ja/ko…）强制指定回答语言。
    group_type: 同行关系（adult/family_kids/senior/couple），切换叙事人格。
    返回: { reply, emotion, sources, context_used, intent }
    """
    history = history or []
    # 按 人格 + 语言 隔离缓存，避免跨人格/跨语种串答
    cache_query = f"{group_type}::{lang}::{query}"
    intent = classify(query, history)  # Orchestrator 分派：决定走哪条处理链路
    fast = fast or intent in _FAST_INTENTS  # 生成性意图自动提速（qwen-turbo）
    # 路线/推荐含实时客流（随时段变），不缓存，避免冻结错峰建议
    cacheable = not history and intent not in _CROWD_INTENTS
    # ── 缓存命中：无历史时才缓存（历史对话有上下文，不能复用） ──────────
    if cacheable:
        cached = answer_cache.get(scenic_id, cache_query)
        if cached:
            save_conversation(session_id, scenic_id, query,
                              cached["reply"], cached["emotion"], cached["context_used"])
            return {**cached, "from_cache": True}
    cfg = get_scenic_config(scenic_id)
    if skip_retrieval(intent):       # 闲聊无需检索知识库，省一次检索延迟
        context_chunks = []
    else:
        retrieval_query = rewrite_query(query, history)  # 改写后用于检索，原 query 用于 LLM
        context_chunks = retrieve(retrieval_query, scenic_id)
    context_used = len(context_chunks) > 0
```

```
    system_prompt = build_system_prompt(
        cfg["name"], cfg["guide_name"], context_chunks, lang,
        _directive_with_context(intent, scenic_id), group_type,
    )
    raw_reply = chat(system_prompt, history, query, fast=fast)
    reply, emotion = extract_emotion(raw_reply)
    result = {
        "reply": reply,
        "emotion": emotion,
        "sources": list({c.split("|")[0].strip() for c in context_chunks}),
        "context_used": context_used,
        "intent": intent.value,
        "scenic_id": scenic_id,
        "from_cache": False,
    }
    if cacheable:
        answer_cache.set(scenic_id, cache_query, result)
    save_conversation(session_id, scenic_id, query, reply, emotion, context_used)
    return result
def _split_to_sentences(text: str) -> list[str]:
    """把纯文本按中文句子结束符切分，用于缓存命中时的流式模拟。"""
    enders = frozenset("。！？…!?")
    sentences, current = [], ""
    for ch in text:
        current += ch
        if ch in enders:
            if current.strip():
                sentences.append(current.strip())
            current = ""
    if current.strip():
        sentences.append(current.strip())
    return sentences or [text]
async def answer_stream(
    query: str,
    scenic_id: str,
    history: list[dict] | None = None,
    session_id: str = "anonymous",
    fast: bool = False,
    lang: str = "auto",
    group_type: str = "adult",
):
    """
    分句流式版本，yield 结构化事件 dict（供 SSE 下发给数字人驱动端）：
      {"type": "emotion", "value": "happy"}
      {"type": "sentence", "text": "...", "index": N}
      {"type": "end",     "full_text": "..."}
    第一个事件（emotion）在 LLM 输出第一个 token 后毫秒级下发，
    数字人表情无需等待全文生成完毕。
    lang: "auto"（跟随提问语言）或语言代码（en/ja/ko…）强制指定回答语言。
    """
```

```
history = history or []
cache_query = f"{group_type}::{lang}::{query}"  # 按 人格+语言 隔离缓存
intent = classify(query, history)  # 先分派；无历史时纯规则匹配，0ms
fast = fast or intent in _FAST_INTENTS  # 生成性意图自动提速（qwen-turbo）
cacheable = not history and intent not in _CROWD_INTENTS  # 含实时客流的不缓存
def _intent_event() -> dict:
    return {"type": "intent", "value": intent.value, "label": INTENT_LABELS[intent]}
# ── 缓存命中：模拟分句流，接口与正常流保持一致 ──────────────────────────
if cacheable:
    cached = answer_cache.get(scenic_id, cache_query)
    if cached:
        save_conversation(session_id, scenic_id, query,
                          cached["reply"], cached["emotion"], cached["context_used"])
        yield _intent_event()
        yield {"type": "emotion", "value": cached["emotion"]}
        for i, s in enumerate(_split_to_sentences(cached["reply"])):
            yield {"type": "sentence", "text": s, "index": i}
        yield {"type": "end", "full_text": cached["reply"]}
        return
cfg = get_scenic_config(scenic_id)
if skip_retrieval(intent):       # 闲聊跳过检索
    context_chunks = []
else:
    retrieval_query = await asyncio.to_thread(rewrite_query, query, history)
    if settings.deploy_mode == "cloud":
        context_chunks = await retrieve_async(retrieval_query, scenic_id)
    else:
        context_chunks = await asyncio.to_thread(retrieve, retrieval_query, scenic_id)
context_used = len(context_chunks) > 0
directive = await asyncio.to_thread(_directive_with_context, intent, scenic_id)
system_prompt = build_system_prompt(
    cfg["name"], cfg["guide_name"], context_chunks, lang,
    directive, group_type,
)
yield _intent_event()  # intent 在 LLM 调用前即可下发，比 emotion 更早
full_text = ""
emotion = "neutral"
async for event in chat_stream(system_prompt, history, query, fast=fast):
    if event["type"] == "emotion":
        emotion = event["value"]
    elif event["type"] == "end":
        full_text = event["full_text"]
    yield event  # 原样透传给 API 层
reply, _ = extract_emotion(full_text)
if cacheable:
    answer_cache.set(scenic_id, cache_query, {
        "reply": reply, "emotion": emotion,
        "sources": list({c.split("|")[0].strip() for c in context_chunks}),
        "context_used": context_used, "intent": intent.value, "scenic_id": scenic_id,
    })
```

```
    save_conversation(session_id, scenic_id, query, reply, emotion, context_used)
==================== backend/rag/indexer.py ====================
"""
知识库入库：把文档分块、向量化、存入对应景区的 ChromaDB collection。
"""
from pathlib import Path
from typing import List
import uuid
import chromadb
from backend.rag.loader import load_docx_as_chunks
from backend.rag.retriever import get_collection
from backend.core.config import settings
def _embed(texts):
    if settings.deploy_mode == "cloud":
        from backend.rag.embedder_dashscope import embed_texts_cloud
        return embed_texts_cloud(texts)
    from backend.rag.embedder import embed_texts
    return embed_texts(texts)
def index_documents(file_paths: List[Path], scenic_id: str) -> int:
    """
    将文档列表入库到指定景区的 collection。
    返回本次入库的 chunk 总数。
    """
    collection = get_collection(scenic_id)
    all_chunks: List[dict] = []
    for path in file_paths:
        if path.suffix.lower() == ".docx":
            chunks = load_docx_as_chunks(
                path,
                chunk_size=settings.chunk_size,
                overlap=settings.chunk_overlap,
            )
            all_chunks.extend(chunks)
    if not all_chunks:
        return 0
    texts = [c["text"] for c in all_chunks]
    embeddings = _embed(texts)
    collection.add(
        ids=[str(uuid.uuid4()) for _ in texts],
        documents=texts,
        embeddings=embeddings,
        metadatas=[{"source": c["source"]} for c in all_chunks],
    )
    return len(texts)
def clear_scenic(scenic_id: str) -> None:
    """清空某景区的知识库（重新上传时先清空）。"""
    from backend.rag.retriever import invalidate_entity_cache
    client = chromadb.PersistentClient(path=str(settings.vector_store_dir))
    try:
        client.delete_collection(f"scenic_{scenic_id}")
```

```
    except Exception:
        pass
    invalidate_entity_cache(scenic_id)  # EA-RAG 景点名缓存随知识库失效
==================== backend/rag/loader.py ====================
"""
文档加载与分块。
两种文档类型处理策略不同：
- 叙述性文档（游览指南）：按段落+滑动窗口分块，保留语义连贯性
- 结构化表格（景点数据集）：每行作为一个独立 chunk，保证字段完整
"""
from pathlib import Path
from typing import List
from docx import Document as DocxDocument
def _load_docx_paragraphs(path: Path) -> List[str]:
    doc = DocxDocument(str(path))
    paragraphs = [p.text.strip() for p in doc.paragraphs if p.text.strip()]
    # 提取表格内容
    for table in doc.tables:
        headers = [cell.text.strip() for cell in table.rows[0].cells]
        for row in table.rows[1:]:
            cells = [cell.text.strip() for cell in row.cells]
            if any(cells):
                # 把每行拼成"字段名: 值"格式，方便语义检索
                row_text = " | ".join(
                    f"{h}: {v}" for h, v in zip(headers, cells) if v
                )
                paragraphs.append(row_text)
    return paragraphs
def chunk_paragraphs(
    paragraphs: List[str],
    chunk_size: int = 512,
    overlap: int = 128,
) -> List[str]:
    """把段落列表合并成目标 token 量的 chunk，相邻 chunk 有重叠。"""
    chunks: List[str] = []
    current = ""
    for para in paragraphs:
        if len(current) + len(para) <= chunk_size:
            current = current + "\n" + para if current else para
        else:
            if current:
                chunks.append(current.strip())
            # 用上一个 chunk 的末尾作为重叠部分
            overlap_text = current[-overlap:] if len(current) > overlap else current
            current = overlap_text + "\n" + para if overlap_text else para
    if current.strip():
        chunks.append(current.strip())
    return chunks
def load_docx_as_chunks(
    path: Path,
```

```
    chunk_size: int = 512,
    overlap: int = 128,
) -> List[dict]:
    """加载 docx，返回 [{text, source}] 列表。"""
    paragraphs = _load_docx_paragraphs(path)
    chunks = chunk_paragraphs(paragraphs, chunk_size, overlap)
    return [{"text": c, "source": path.name} for c in chunks]
==================== backend/rag/embedder.py ====================
"""
本地 BGE-M3 嵌入模型封装。
BGE-M3 同时支持稠密向量、稀疏向量、多向量，这里只用稠密向量（dense）。
首次运行会从 HuggingFace 下载模型（约 2.2GB），之后缓存在本地。
"""
import os
from typing import List
os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
for _var in ("ALL_PROXY", "all_proxy", "HTTPS_PROXY", "https_proxy",
             "HTTP_PROXY", "http_proxy"):
    os.environ.pop(_var, None)
from huggingface_hub import snapshot_download
from FlagEmbedding import BGEM3FlagModel
from backend.core.config import settings
_model: BGEM3FlagModel | None = None
def _ensure_model_downloaded(model_name: str) -> str:
    """下载模型到本地缓存，跳过会导致 403 的无关文件。"""
    local_path = snapshot_download(
        repo_id=model_name,
        ignore_patterns=["*.DS_Store", "imgs/*", "*.h5", "*.ot", "flax_model*", "rust_model*", "tf_model*"],
    )
    return local_path
def get_embedder() -> BGEM3FlagModel:
    global _model
    if _model is None:
        local_path = _ensure_model_downloaded(settings.embedding_model)
        _model = BGEM3FlagModel(
            local_path,
            use_fp16=True,
        )
    return _model
def embed_texts(texts: List[str]) -> List[List[float]]:
    """返回归一化后的稠密向量列表，可直接存入 ChromaDB。"""
    model = get_embedder()
    result = model.encode(
        texts,
        batch_size=32,
        max_length=512,
        return_dense=True,
        return_sparse=False,
        return_colbert_vecs=False,
    )
```

```
    return result["dense_vecs"].tolist()
==================== backend/rag/embedder_dashscope.py ====================
"""
DashScope 云端嵌入（部署到阿里云时使用）。
text-embedding-v3 是通义千问的向量嵌入模型，效果接近 BGE-M3，无需 GPU。
"""
import os
from typing import List
import dashscope
from dashscope import TextEmbedding
def embed_texts_cloud(texts: List[str]) -> List[List[float]]:
    """批量嵌入，DashScope 单次最多 25 条，自动分批。"""
    all_vecs = []
    batch_size = 10
    for i in range(0, len(texts), batch_size):
        batch = texts[i: i + batch_size]
        resp = TextEmbedding.call(
            model="text-embedding-v3",
            input=batch,
        )
        vecs = [item["embedding"] for item in resp.output["embeddings"]]
        all_vecs.extend(vecs)
    return all_vecs
==================== backend/rag/cache.py ====================
"""
LRU 答案缓存。
命中条件：scenic_id + query 完全一致（忽略首尾空格和大小写）。
缓存命中直接返回，跳过 embed + 检索 + LLM，延迟从 ~2s → <5ms。
"""
import hashlib
import time
from collections import OrderedDict
from typing import Any
class LRUCache:
    def __init__(self, maxsize: int = 300, ttl_seconds: int = 3600):
        self._cache: OrderedDict[str, tuple[Any, float]] = OrderedDict()
        self.maxsize = maxsize
        self.ttl = ttl_seconds
    def _key(self, scenic_id: str, query: str) -> str:
        normalized = f"{scenic_id}::{query.strip().lower()}"
        return hashlib.md5(normalized.encode()).hexdigest()
    def get(self, scenic_id: str, query: str) -> Any | None:
        key = self._key(scenic_id, query)
        if key not in self._cache:
            return None
        value, ts = self._cache[key]
        if time.time() - ts > self.ttl:
            del self._cache[key]
            return None
        self._cache.move_to_end(key)
```

```
        return value
    def set(self, scenic_id: str, query: str, value: Any) -> None:
        key = self._key(scenic_id, query)
        self._cache[key] = (value, time.time())
        self._cache.move_to_end(key)
        if len(self._cache) > self.maxsize:
            self._cache.popitem(last=False)
    def invalidate(self, scenic_id: str | None = None) -> None:
        """知识库更新后调用，清空对应景区缓存。"""
        if scenic_id is None:
            self._cache.clear()
            return
        drop = [k for k, (v, _) in self._cache.items()
                if isinstance(v, dict) and v.get("scenic_id") == scenic_id]
        for k in drop:
            del self._cache[k]
    @property
    def size(self) -> int:
        return len(self._cache)
answer_cache = LRUCache(maxsize=300, ttl_seconds=3600)
==================== backend/rag/streaming.py ====================
"""
流式 RAG 流水线：LLM 生成第一句话就立即推送给 TTS，不等完整回复。
时序图：
  LLM:  [句1生成完]→[句2生成中...]→[句3生成中...]
  TTS:      [句1合成]→[句2合成]→[句3合成]
  数字人:       [句1播放]→[句2播放]→[句3播放]
这样用户听到第一句话的延迟 ≈ RAG检索 + LLM首句生成 + TTS首句合成
≈ 0.3s + 0.8s + 0.4s = 1.5s（GPU状态下）
"""
import re
import asyncio
from typing import AsyncGenerator
from backend.rag.retriever import retrieve
from backend.llm.prompts import build_system_prompt
from backend.llm.client import chat_stream
from backend.rag.pipeline import get_scenic_config
# 句子分隔符：遇到这些符号就认为一句话结束，立即送给 TTS
_SENTENCE_END = re.compile(r'([。 ！ ？!?…]+|\.{2,})')
async def stream_sentences(
    query: str,
    scenic_id: str,
    history: list[dict] | None = None,
) -> AsyncGenerator[dict, None]:
    """
    按句子流式输出，每完成一句就 yield 一次。
    队友的 TTS 模块订阅这个流，每收到一句就立即合成音频。
    yield 格式:
      {"type": "sentence", "text": "这是一句话"}
      {"type": "done", "full_text": "完整回复"}
```

```
    """
    history = history or []
    cfg = get_scenic_config(scenic_id)
    context_chunks = retrieve(query, scenic_id)
    system_prompt = build_system_prompt(
        scenic_name=cfg["name"],
        guide_name=cfg["guide_name"],
        context_chunks=context_chunks,
    )
    buffer = ""
    full_text = ""
    async for token in chat_stream(system_prompt, history, query):
        buffer += token
        full_text += token
        # 检测句子边界
        match = _SENTENCE_END.search(buffer)
        while match:
            end_pos = match.end()
            sentence = buffer[:end_pos].strip()
            if sentence:
                yield {"type": "sentence", "text": sentence}
            buffer = buffer[end_pos:]
            match = _SENTENCE_END.search(buffer)
    # 输出剩余不完整的句子
    if buffer.strip():
        yield {"type": "sentence", "text": buffer.strip()}
    yield {"type": "done", "full_text": full_text}
==================== backend/agent/orchestrator.py ====================
"""
调度中枢 Orchestrator：意图识别 + 路由决策。
混合策略（速度与准确兼顾）：
  1. 规则快筛：高频关键词秒级命中，0ms（见 intents.py）
  2. LLM 兜底：规则未命中、且问题模糊（短问题 + 有上下文）时，
     调 qwen-turbo 分类；否则默认走知识问答 qa。
这是把"多智能体"从架构叙事落地为真实代码的核心模块：
classify() 决定每轮请求交给哪条处理逻辑（见 handlers.py）。
"""
from backend.agent.intents import Intent, match_by_rules
from backend.llm.client import chat
_CLASSIFY_PROMPT = """你是景区导览系统的意图分类器。判断游客这句话属于以下哪一类，只输出一个英文标签，不要任何
    解释：
- qa：询问景点的历史、文化、数据等知识性问题
- navigation：询问游览路线、顺序、行程安排、怎么走
- recommend：寻求推荐（值得去的、适合的、拍照点、好吃的等）
- empathy：表达负面情绪、抱怨、求助、遇到困难
- chitchat：与景点无关的闲聊、问候、调侃
游客的话：{query}
标签："""
_VALID = {i.value for i in Intent}
def _classify_by_llm(query: str) -> Intent:
```

```
    """LLM 兜底分类。失败或输出异常时安全回退到 qa。"""
    try:
        label = chat(_CLASSIFY_PROMPT.format(query=query), [], "", fast=True).strip().lower()
        for v in _VALID:        # 容错：模型可能带多余字符，取首个合法标签
            if v in label:
                return Intent(v)
    except Exception:
        pass
    return Intent.QA
def _is_ambiguous(query: str, history: list[dict]) -> bool:
    """短问题 + 有上下文 → 模糊，值得 LLM 兜底；否则默认 qa 已足够。"""
    return len(query.strip()) < 8 and bool(history)
def classify(query: str, history: list[dict] | None = None) -> Intent:
    """
    返回本轮请求的意图。
    规则命中 → 直接返回（0ms）；未命中且模糊 → LLM 兜底；否则默认 qa。
    """
    history = history or []
    hit = match_by_rules(query)
    if hit is not None:
        return hit
    if _is_ambiguous(query, history):
        return _classify_by_llm(query)
    return Intent.QA
==================== backend/agent/intents.py ====================
"""
意图定义与关键词规则表。
Orchestrator 的"快筛"层：高频关键词秒级命中即路由（0ms），
未命中再交给 LLM 兜底（见 orchestrator.py）。
"""
from enum import Enum
class Intent(str, Enum):
    QA       = "qa"        # 知识问答（默认，走 RAG）
    NAVIGATION = "navigation"  # 路线规划
    RECOMMEND  = "recommend"   # 个性化推荐
    EMPATHY    = "empathy"     # 情感安抚 / 求助
    CHITCHAT   = "chitchat"    # 闲聊（跳过 RAG）
# 中文意图名（演示、日志、前端展示用）
INTENT_LABELS: dict["Intent", str] = {
    Intent.QA:       "知识问答",
    Intent.NAVIGATION: "路线规划",
    Intent.RECOMMEND:  "个性推荐",
    Intent.EMPATHY:    "情感安抚",
    Intent.CHITCHAT:   "闲聊互动",
}
# 关键词规则表。按列表顺序匹配，第一个命中的意图胜出。
# 顺序很重要：先判信号最强、最该优先处理的意图（情感安抚），
# 再到路线、推荐，最后闲聊。
_RULES: list[tuple[Intent, tuple[str, ...]]] = [
    (Intent.EMPATHY, (
```

```
        "排队", "太久", "好烦", "烦死", "累死", "好累", "投诉", "不满",
        "退票", "退钱", "找不到", "迷路", "怎么办", "帮帮", "太差",
        "失望", "生气", "差评", "态度",
    )),
    (Intent.NAVIGATION, (
        "怎么走", "怎么去", "怎么逛", "路线", "顺序", "先去", "导航", "线路",
        "行程", "攻略", "几个小时", "半天", "一天", "时间安排", "先逛", "路过",
        "从哪", "到哪",
    )),
    (Intent.RECOMMEND, (
        "推荐", "值得", "必去", "必看", "有什么好", "适合", "哪个好", "最好玩",
        "不容错过", "打卡", "拍照", "出片", "好吃的", "特产",
    )),
    (Intent.CHITCHAT, (
        "你叫什么", "你是谁", "你几岁", "你多大", "你会", "你能做", "你好呀",
        "无聊", "陪我", "讲个笑话", "唱歌", "你喜欢",
    )),
]
def match_by_rules(query: str) -> Intent | None:
    """关键词快筛。命中返回对应意图，未命中返回 None（交给 LLM 兜底）。"""
    for intent, keywords in _RULES:
        if any(kw in query for kw in keywords):
            return intent
    return None
==================== backend/agent/handlers.py ====================
"""
各意图的处理策略：为不同意图注入不同的"任务指令"到 system prompt。
qa 走默认行为（指令为空）；其余意图追加针对性的导游话术要求。
这样同一套 RAG + LLM 链路，能对不同诉求给出差异化、有温度的回应。
"""
from backend.agent.intents import Intent
_DIRECTIVES: dict[Intent, str] = {
    Intent.NAVIGATION: (
        "\n【本轮任务：路线规划】游客在咨询游览路线。请结合知识库中的路线/景点信息，"
        "给出清晰的游览顺序，包含：建议路线、各点停留时间、景点之间的衔接讲解。"
        "若游客提到时间（半天/一天）或同行情况，据此调整路线松紧。\n"
    ),
    Intent.RECOMMEND: (
        "\n【本轮任务：个性化推荐】游客在寻求推荐。请基于知识库内容推荐 2-4 个景点/活动，"
        "每条说明推荐理由（有什么特别）。若游客提到偏好（拍照/亲子/文化），优先匹配。\n"
    ),
    Intent.EMPATHY: (
        "\n【本轮任务：情感安抚】游客可能情绪不佳或遇到困难。请首先共情、安抚，语气温暖耐心，"
        "再给出实际可行的帮助或建议。情绪标签优先选 [情绪:关切]。\n"
    ),
    Intent.CHITCHAT: (
        "\n【本轮任务：闲聊互动】游客在和你闲聊。请以亲切活泼的导游人格自然回应，简短友好，"
        "适度把话题引回景区游览，但不必生硬。无需引用知识库。\n"
    ),
}
```

```
def intent_directive(intent: Intent) -> str:
    """返回意图对应的任务指令（注入 system prompt）。qa 返回空字符串。"""
    return _DIRECTIVES.get(intent, "")
def skip_retrieval(intent: Intent) -> bool:
    """闲聊无需检索知识库，省一次检索延迟。"""
    return intent == Intent.CHITCHAT
==================== backend/llm/prompts.py ====================
"""
Prompt 模板。
设计原则：
1. 景区名称动态注入，不硬编码"灵山"
2. 明确指示模型只用提供的知识回答，拒绝幻觉
3. 情绪化、亲切的导游人格，提升游客体验分
4. 无法回答时的标准兜底话术，避免胡说
"""
SYSTEM_PROMPT_TEMPLATE = """你是{scenic_name}的专属AI导游，名叫"{guide_name}"。
你热情亲切、知识渊博，擅长用生动的语言介绍景区的历史、文化和景点特色。
【情绪标注】（必须在正文第一句话之前输出）
根据本次回答基调，在开始正文之前先单独输出一个情绪标签，格式固定，只选一个：
[情绪:开心] 或 [情绪:兴奋] 或 [情绪:平静] 或 [情绪:好奇] 或 [情绪:关切] 或 [情绪:抱歉]
选择依据：正面讲解→开心；描述震撼景点→兴奋；客观陈述→平静；引导互动→好奇；游客困惑时→关切；无法回答时→抱歉。
输出情绪标签后，换行开始正文。
【回答规则】
1. 只根据下方【景区知识库】中的内容回答，不要凭空捏造数据或信息
2. 回答要自然流畅，像真人导游讲解，不要像在背课文
3. 如果问题与景区无关，礼貌地引导游客回到景区话题
4. 如果知识库中没有足够信息，诚实说"这个问题我需要进一步查阅，您可以向工作人员咨询"
5. 涉及价格、时间等易变信息时，提醒游客以景区现场公告为准
6. 【口播格式·重要】你的回答会被数字人**朗读**出来，必须是连贯口语：
  - 禁止任何 Markdown 标记（#、*、**、-、`、表格、代码块）和"###""**加粗**"这类符号
  - 不要用"第一/第二/1./2."式的硬分点或编号清单
  - 改用"先……再……接着……最后……""沿途您会经过……"等口语过渡，把内容自然串成段落，像真人导游边走边讲
7. 【开口要快·重要】正文的**第一句必须简短**（15 字以内、以句号结尾），并直接点出问题的核心答案，之后再展开细
   节。
  例如问"大佛多高"→先说"灵山大佛高 88 米。"再补充；问"几点表演"→先说"九龙灌浴每天有四场。"再说具体时间。
  绝不要用"……真是""这个问题嘛""说到……"这类冗长铺垫开头——数字人要靠第一句立刻开口，开头越短越好。
{persona_directive}{task_directive}{lang_directive}
【景区知识库】
{context}
"""
# 语言代码 → 名称（用于强制指定回答语言）
LANG_NAMES = {
    "en": "English",
    "ja": "日本語",
    "ko": "한국어",
    "fr": "Français",
    "de": "Deutsch",
    "es": "Español",
    "ru": "Русский",
    "zh": "简体中文",
```

```
}
# 同行关系 → 叙事人格（情境感知：同样的景点，讲给不同人方式截然不同）
GROUP_LABELS = {
  "adult":     "通用成人",
  "family_kids": "亲子家庭",
  "senior":    "老年长辈",
  "couple":    "情侣出游",
}
_PERSONA_DIRECTIVES = {
  "family_kids": (
    "\n【叙事人格：亲子家庭】同行有小朋友。请多用讲故事的方式，把历史文化拟人化、"
    "趣味化，适当抛出互动小问题（“小朋友猜猜看～”），并主动提示亲子设施、"
    "洗手间、母婴室的位置。语气活泼，避免生僻术语。\n"
  ),
  "senior": (
    "\n【叙事人格：老年长辈】同行有长辈。请放慢叙述节奏，句子简短清晰，"
    "重点提示无障碍通道、休息座椅、医疗点、缓坡路线，避免推荐需大量爬山或久站的项目。"
    "语气温和耐心，多一份关怀。\n"
  ),
  "couple": (
    "\n【叙事人格：情侣出游】同行是情侣。请用浪漫、有意境的视角讲解，"
    "主动推荐适合合影的最佳机位、安静私密的角落、有美好寓意的祈福点和日落观景处。"
    "语气温馨甜美。\n"
  ),
}
def _build_persona_directive(group_type: str) -> str:
  """生成同行人格指令。adult 或未知值返回空串（默认通用人格）。"""
  return _PERSONA_DIRECTIVES.get(group_type, "")
def _build_lang_directive(lang: str) -> str:
  """生成回答语言指令。情绪标签始终保持中文固定格式，仅正文切换语言。"""
  if lang == "auto":
    return (
      "\n【回答语言】请用游客提问所使用的语言回答正文"
      "（中文提问用中文答，English question → answer in English）。"
      "但开头的情绪标签 [情绪:X] 必须始终保持上述中文固定格式，不要翻译它。\n"
    )
  name = LANG_NAMES.get(lang)
  if not name:
    return ""
  return (
    f"\n【回答语言】无论游客用什么语言提问，正文一律用 {name} 回答。"
    f"但开头的情绪标签 [情绪:X] 必须保持上述中文固定格式，不要翻译它。\n"
  )
QUERY_REWRITE_PROMPT = """你是一个搜索查询优化专家。将游客的口语问题改写为更适合向量检索的标准形式。
改写规则：
1. 补全指代词（"它"→具体景点名，"这里"→景区名称）
2. 口语转书面（"多高啊"→"高度是多少米"）
3. 补全省略的主语
4. 保留所有关键词，不要删减信息
5. 只输出改写后的问题，不要任何解释
```

```
对话历史（用于理解指代关系）：
{history}
当前问题：{query}
改写后："""
def needs_rewrite(query: str, history: list[dict]) -> bool:
    """判断是否需要改写：含指代词或历史中有上下文时才改写，避免浪费 API。"""
    pronouns = {"它", "这", "那", "这里", "那里", "上面", "之前", "前面", "刚才", "其", "该"}
    if any(p in query for p in pronouns):
        return True
    if len(query.strip()) < 6 and history:
        return True
    return False
def build_system_prompt(
    scenic_name: str,
    guide_name: str,
    context_chunks: list[str],
    lang: str = "auto",
    task_directive: str = "",
    group_type: str = "adult",
) -> str:
    context = "\n\n---\n\n".join(context_chunks) if context_chunks else "（暂无相关资料）"
    return SYSTEM_PROMPT_TEMPLATE.format(
        scenic_name=scenic_name,
        guide_name=guide_name,
        context=context,
        persona_directive=_build_persona_directive(group_type),
        task_directive=task_directive,
        lang_directive=_build_lang_directive(lang),
    )
==================== backend/llm/client.py ====================
"""
通义千问 API 封装（DashScope）。
两个接口：
- chat()：文本对话，用 qwen-max-latest
- chat_stream()：流式输出，前端打字机效果用
"""
import re
from typing import AsyncGenerator, List
import dashscope
from dashscope import Generation
from backend.core.config import settings
dashscope.api_key = settings.dashscope_api_key
# 中文情绪标签 → 英文 key（数字人表情控制用）
_EMOTION_MAP = {
    "开心": "happy",
    "兴奋": "excited",
    "平静": "neutral",
    "好奇": "curious",
    "关切": "empathetic",
    "抱歉": "apologetic",
```

```
}
# 不锚定位置：情绪标签现在在正文前，但也兼容在末尾的旧格式
_EMOTION_RE = re.compile(
    r'\[情绪[：:]\s*(开心|兴奋|平静|好奇|关切|抱歉)\s*\]'
)
# 中文句子结束符
_SENTENCE_ENDERS = frozenset('。！？…!?')
def extract_emotion(text: str) -> tuple[str, str]:
    """从文本中解析 [情绪:X] 标签（可在开头或结尾）。返回 (干净正文, emotion_key)。"""
    m = _EMOTION_RE.search(text)
    if m:
        clean = (text[:m.start()] + text[m.end():]).strip()
        return clean, _EMOTION_MAP[m.group(1)]
    return text.strip(), "neutral"
def _get_model(fast: bool) -> str:
    """fast=True 用 qwen-turbo（首 token ~150ms），否则 qwen-max（质量更高）。"""
    return "qwen-turbo" if fast else settings.llm_model
def chat(
    system_prompt: str,
    history: List[dict],
    user_message: str,
    fast: bool = False,
) -> str:
    """同步调用，返回完整回复文本。fast=True 延迟减半。"""
    messages = [{"role": "system", "content": system_prompt}]
    messages.extend(history)
    messages.append({"role": "user", "content": user_message})
    response = Generation.call(
        model=_get_model(fast),
        messages=messages,
        result_format="message",
        temperature=0.7,
        max_tokens=1024,
    )
    if response.status_code != 200 or response.output is None:
        raise RuntimeError(
            f"DashScope 调用失败 [{response.status_code} {response.code}]: {response.message}"
        )
    return response.output.choices[0].message.content
async def _raw_token_stream(
    system_prompt: str,
    history: List[dict],
    user_message: str,
    fast: bool = False,
) -> AsyncGenerator[str, None]:
    """底层 token 流，供内部使用。"""
    import asyncio
    import threading
    messages = [{"role": "system", "content": system_prompt}]
    messages.extend(history)
```

```
    messages.append({"role": "user", "content": user_message})
    # DashScope 的流式是同步阻塞 generator；若直接在 async 里 for 迭代会卡住
    # 事件循环，导致 FastAPI 无法逐块 flush SSE（表现为整段答案一次性到达）。
    # 解决：同步迭代放到线程里跑，token 经 asyncio.Queue 异步交回，事件循环不被阻塞。
    queue: asyncio.Queue = asyncio.Queue()
    loop = asyncio.get_running_loop()
    def _produce():
        try:
            responses = Generation.call(
                model=_get_model(fast),
                messages=messages,
                result_format="message",
                stream=True,
                incremental_output=True,
                temperature=0.7,
                max_tokens=1024,
            )
            for resp in responses:
                if resp.status_code != 200 or resp.output is None:
                    loop.call_soon_threadsafe(
                        queue.put_nowait,
                        ("error", f"DashScope 流式失败 [{resp.status_code} {resp.code}]: {resp.message}"),
                    )
                    return
                delta = resp.output.choices[0].message.content
                if delta:
                    loop.call_soon_threadsafe(queue.put_nowait, ("token", delta))
        except Exception as e:  # noqa: BLE001
            loop.call_soon_threadsafe(queue.put_nowait, ("error", str(e)))
        finally:
            loop.call_soon_threadsafe(queue.put_nowait, ("done", None))
    threading.Thread(target=_produce, daemon=True).start()
    while True:
        kind, val = await queue.get()
        if kind == "done":
            break
        if kind == "error":
            raise RuntimeError(val)
        yield val
async def chat_stream(
    system_prompt: str,
    history: List[dict],
    user_message: str,
    fast: bool = False,
) -> AsyncGenerator[dict, None]:
    """
    分句流式调用，yield 结构化事件 dict，供数字人驱动端使用：
      {"type": "emotion", "value": "happy"}
          — 第一个事件，情绪已知，数字人立即设置表情
      {"type": "sentence", "text": "你好！欢迎来到灵山胜境～", "index": 0}
```

```
    — 每完整一句 yield 一次，TTS 收到即可开始合成
  {"type": "end",   "full_text": "...（含情绪标签的原始输出）"}
    — 流结束，pipeline 用 full_text 做缓存/存库
  """
  buffer = ""
  full_text = ""
  sentence_idx = 0
  emotion_found = False
  async for token in _raw_token_stream(system_prompt, history, user_message, fast):
    buffer += token
    full_text += token
    # ── 优先探测情绪标签（出现即立刻 yield，不等到句子结束）──────────────
    if not emotion_found:
      m = _EMOTION_RE.search(buffer)
      if m:
        emotion_found = True
        yield {"type": "emotion", "value": _EMOTION_MAP[m.group(1)]}
        # 从 buffer 中移除标签，剩余内容继续分句处理
        buffer = (buffer[:m.start()] + buffer[m.end():]).lstrip("\n").lstrip()
    # ── 扫描 buffer，遇句子结束符就 yield 完整一句 ──────────────────────────
    while True:
      end_pos = next(
        (i for i, ch in enumerate(buffer) if ch in _SENTENCE_ENDERS), -1
      )
      if end_pos == -1:
        break
      sentence = buffer[: end_pos + 1].strip()
      buffer = buffer[end_pos + 1 :]
      if sentence:
        yield {"type": "sentence", "text": sentence, "index": sentence_idx}
        sentence_idx += 1
  # ── 流结束：处理 buffer 尾部 ──────────────────────────────────────────────
  # 兜底：LLM 未遵循"在前"指令，情绪标签出现在末尾
  if not emotion_found:
    m = _EMOTION_RE.search(buffer)
    if m:
      yield {"type": "emotion", "value": _EMOTION_MAP[m.group(1)]}
      buffer = (buffer[:m.start()] + buffer[m.end():]).strip()
  # 最后一句（无结束符的尾部文字）
  if buffer.strip():
    yield {"type": "sentence", "text": buffer.strip(), "index": sentence_idx}
  yield {"type": "end", "full_text": full_text}
==================== backend/llm/multimodal.py ====================
"""
多模态问答：qwen-vl-max。
满足赛题"至少使用 1 个多模态大模型"的硬性要求。
不只是"看图答题"，而是**视觉定位 + 口语指路**的随身导游（创新三 / AR-lite）：
游客举起手机拍下眼前景物 → 模型像真人导游一样：
 ① 定位：判断这是本景区哪个地标，先告诉游客"您正站在 X"
 ② 讲解：以会讲故事的导游口吻介绍文化背景与看点
```

```
③ 指路：主动推荐邻近景点并给方向性指引（往…方向，步行约几分钟）
程序解析两个标签：[位置:X]（给前端显示 ☒ 当前位置）、[情绪:X]（驱动数字人表情）。
"""
import base64
import re
from pathlib import Path
import dashscope
from dashscope import MultiModalConversation
from backend.core.config import settings
from backend.llm.client import extract_emotion
dashscope.api_key = settings.dashscope_api_key
# [位置:X] 标签：X 为识别到的景区地标名；未识别景区地标时为"未识别"
_LOCATION_RE = re.compile(r'\[位置[：:]\s*([^\]]+?)\s*\]')
def _build_vl_system(scenic_name: str) -> str:
    return f"""你是{scenic_name}的随身AI导游。游客此刻**人就在{scenic_name}景区内**，举起手机拍下了眼前的景物
    ，请像一位热情、会讲故事的真人导游那样，按下面的顺序回答：
① 定位：判断画面里是{scenic_name}的哪个地标或景点。
  - 游客就在本景区，所以只要画面像是{scenic_name}的景物，就优先判断它是本景区的哪个具体景点，第一句明确告诉游
    客所在位置，例如"您现在正站在大佛广场，眼前就是灵山大佛"。
  - 只有当画面明显不是景区景物（随手拍的人、食物、天空，或一眼看出是别处的地标）时，才说明暂时无法据此定位，
    并礼貌引导游客对准景点再拍。
② 讲解：用生动、会讲故事的导游口吻，介绍这个景点的文化背景、最值得看的地方，再讲一个有意思的小故事或细节，别干
    巴巴罗列事实。
③ 指路：主动为游客推荐接下来最值得去的 1~2 个邻近景点，并给出方向性指引。
  方向用"往……方向，步行大约几分钟"这种相对描述即可；不要编造精确的米数或门牌号。
严格遵守以下格式（供程序解析，游客看不到标签本身）：
- 正文最前面单独一行标注识别到的位置：[位置:大佛广场]；若无法识别为景区地标，则写 [位置:未识别]。
- 正文最后单独一行标注情绪：[情绪:开心] 或 [情绪:兴奋] 或 [情绪:平静] 或 [情绪:好奇] 或 [情绪:关切] 或 [情绪:
    抱歉]。
中间正文用自然口语连贯成段，不要出现"①②③"这类编号。"""
def _extract_location(text: str) -> tuple[str, str | None]:
    """解析 [位置:X] 标签。返回 (去掉标签的正文, location 或 None)。'未识别' 归一为 None。"""
    m = _LOCATION_RE.search(text)
    if not m:
        return text, None
    loc = m.group(1).strip()
    clean = (text[:m.start()] + text[m.end():]).strip()
    if loc in ("未识别", "无", "未知", ""):
        return clean, None
    return clean, loc
def locate_landmark(
    image_b64: str,
    scenic_name: str = "灵山胜境",
    image_format: str = "jpeg",
) -> str | None:
    """视觉定位（两段式第一段）：只识别这是本景区的哪个地标，返回地标名或 None。
    只让 qwen-vl 做它最擅长的"认地标"，讲解/指路交给 RAG（事实由知识库约束，压幻觉）。
    """
    system = (
        f"游客此刻人在{scenic_name}景区内，拍了这张照片。请只判断这是{scenic_name}的哪个景点。"
```

```
        f"规则：能确定就只输出景点名称（例如：大佛广场），不要任何多余文字、标点或解释；"
        f"若画面明显不是景区景物（人、食物、天空）或一眼看出是别处地标，就只输出两个字：未识别。"
    )
    image_url = f"{image_b64}"
    messages = [
        {"role": "system", "content": [{"text": system}]},
        {"role": "user", "content": [{"image": image_url}, {"text": "这是哪个景点？"}]},
    ]
    response = MultiModalConversation.call(model=settings.vl_model, messages=messages)
    if response.status_code != 200 or response.output is None:
        raise RuntimeError(
            f"DashScope 多模态调用失败 [{response.status_code} {response.code}]: {response.message}"
        )
    raw = response.output.choices[0].message.content[0]["text"].strip()
    # 清掉可能的标点/换行/标签残留，只留景点名
    loc = raw.strip().strip("。．！!，,、 ：:").splitlines()[0].strip() if raw else ""
    if not loc or loc in ("未识别", "无", "未知") or len(loc) > 20:
        return None
    return loc
def answer_with_image(
    image_b64: str,
    question: str,
    scenic_name: str = "灵山胜境",
    image_format: str = "jpeg",
) -> tuple[str, str, str | None]:
    """
    输入 base64 图片 + 文字问题，返回 (reply, emotion, location)。
    location 为识别到的景区地标名（如"大佛广场"），无法定位时为 None。
    image_format: jpeg / png / webp / gif
    """
    image_url = f"{image_b64}"
    messages = [
        {"role": "system", "content": [{"text": _build_vl_system(scenic_name)}]},
        {
            "role": "user",
            "content": [
                {"image": image_url},
                {"text": question or "我面前是什么？请帮我定位、讲解，并指引我接下来去哪。"},
            ],
        },
    ]
    response = MultiModalConversation.call(
        model=settings.vl_model,
        messages=messages,
    )
    if response.status_code != 200 or response.output is None:
        raise RuntimeError(
            f"DashScope 多模态调用失败 [{response.status_code} {response.code}]: {response.message}"
        )
    raw = response.output.choices[0].message.content[0]["text"]
```

```
    # 先剥位置标签，再剥情绪标签
    body, location = _extract_location(raw)
    reply, emotion = extract_emotion(body)
    return reply, emotion, location
def answer_with_image_file(
    image_path: str | Path,
    question: str,
    scenic_name: str = "灵山胜境",
) -> tuple[str, str, str | None]:
    """便捷函数：从本地文件读取图片。"""
    path = Path(image_path)
    fmt = path.suffix.lstrip(".").lower()
    fmt = "jpeg" if fmt in ("jpg", "jpeg") else fmt
    b64 = base64.b64encode(path.read_bytes()).decode()
    return answer_with_image(b64, question, scenic_name, fmt)
==================== backend/api/chat.py ====================
import json
from fastapi import APIRouter
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from backend.rag.pipeline import answer, answer_stream
router = APIRouter(prefix="/chat", tags=["chat"])
class ChatRequest(BaseModel):
    session_id: str = "anonymous"
    message: str
    scenic_id: str = "lingshan"
    history: list[dict] = []
    stream: bool = False
    fast: bool = False     # True → qwen-turbo，首 token 更快
    lang: str = "auto"     # auto=跟随提问语言；或 en/ja/ko/fr… 强制指定
    group_type: str = "adult"  # 同行关系，切换叙事人格：adult/family_kids/senior/couple
class ChatResponse(BaseModel):
    reply: str
    emotion: str          # happy / excited / neutral / curious / empathetic / apologetic
    sources: list[str]
    context_used: bool
    intent: str = "qa"     # qa / navigation / recommend / empathy / chitchat（Orchestrator 分派结果）
    from_cache: bool = False
@router.post("", response_model=ChatResponse)
async def chat_endpoint(req: ChatRequest):
    if req.stream:
        async def event_stream():
            """
            标准 SSE 格式，带命名事件类型，供数字人驱动端订阅：
              event: intent
              data: {"type":"intent","value":"navigation","label":"路线规划"}
              event: emotion
              data: {"type":"emotion","value":"happy"}
              event: sentence
              data: {"type":"sentence","text":"你好！欢迎来到灵山胜境～","index":0}
```

```
        event: end
        data: {"type":"end","full_text":"..."}
      事件顺序：intent（最早，LLM 调用前）→ emotion（首 token 后）→ sentence… → end
      intent 为 Orchestrator 分派结果，前端可用于展示"当前意图"或忽略。
      前端示例（JS）：
        const es = new EventSource('/chat?stream=1')
        es.addEventListener('intent',   e => showIntent(JSON.parse(e.data).label))
        es.addEventListener('emotion',  e => setExpression(JSON.parse(e.data).value))
        es.addEventListener('sentence', e => sendToTTS(JSON.parse(e.data).text))
        es.addEventListener('end',      e => es.close())
      """
      async for event in answer_stream(
        req.message, req.scenic_id, req.history, req.session_id,
        req.fast, req.lang, req.group_type,
      ):
        event_type = event["type"]
        data = json.dumps(event, ensure_ascii=False)
        yield f"event: {event_type}\ndata: {data}\n\n"
    return StreamingResponse(
      event_stream(),
      media_type="text/event-stream",
      headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
  result = answer(
    req.message, req.scenic_id, req.history, req.session_id,
    req.fast, req.lang, req.group_type,
  )
  return ChatResponse(**result)
==================== backend/api/knowledge.py ====================
import shutil
from pathlib import Path
from fastapi import APIRouter, UploadFile, File, Form, HTTPException
import chromadb
from backend.rag.indexer import index_documents, clear_scenic
from backend.core.config import settings
router = APIRouter(prefix="/kb", tags=["knowledge-base"])
@router.post("/upload")
async def upload_documents(
  scenic_id: str = Form(...),
  files: list[UploadFile] = File(...),
  replace: bool = Form(False),
):
  """上传文档并入库。replace=True 时先清空该景区知识库。"""
  save_dir = settings.knowledge_base_dir / scenic_id
  save_dir.mkdir(parents=True, exist_ok=True)
  if replace:
    clear_scenic(scenic_id)
  saved_paths: list[Path] = []
  for file in files:
    dest = save_dir / file.filename
```

```
        with open(dest, "wb") as f:
            shutil.copyfileobj(file.file, f)
        saved_paths.append(dest)
    count = index_documents(saved_paths, scenic_id)
    return {"scenic_id": scenic_id, "files": len(saved_paths), "chunks_indexed": count}
@router.get("/list")
async def list_knowledge_bases():
    """查询所有景区知识库的 chunk 数量。"""
    client = chromadb.PersistentClient(path=str(settings.vector_store_dir))
    result = {}
    for col in client.list_collections():
        scenic_id = col.name.removeprefix("scenic_")
        result[scenic_id] = col.count()
    return result
@router.delete("/{scenic_id}")
async def delete_knowledge_base(scenic_id: str):
    clear_scenic(scenic_id)
    return {"deleted": scenic_id}
==================== backend/api/multimodal.py ====================
"""
多模态聊天接口：游客上传图片提问，用 qwen-vl-max 回答。
"""
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from backend.llm.multimodal import answer_with_image, locate_landmark
from backend.db.database import save_conversation
from backend.rag.pipeline import get_scenic_config, answer as rag_answer
router = APIRouter(prefix="/chat", tags=["multimodal"])
class ImageChatRequest(BaseModel):
    session_id: str = "anonymous"
    scenic_id: str = "lingshan"
    image_b64: str          # base64 编码的图片
    image_format: str = "jpeg"  # jpeg / png / webp
    question: str = "请介绍一下图片里的内容"
    group_type: str = "adult"   # adult/family_kids/senior/couple，切换讲解人格
    lang: str = "auto"
    ground: bool = True       # True=两段式（VL 定位 + RAG 接地讲解，压幻觉）；False=单次 VL
class ImageChatResponse(BaseModel):
    reply: str
    emotion: str
    location: str | None = None   # 识别到的景区地标（如"大佛广场"），无法定位时为 null
    model_used: str = "qwen-vl-max"
@router.post("/image", response_model=ImageChatResponse)
async def image_chat_endpoint(req: ImageChatRequest):
    if not req.image_b64:
        raise HTTPException(status_code=400, detail="image_b64 不能为空")
    cfg = get_scenic_config(req.scenic_id)
    try:
        # 两段式：① VL 只认地标 ② 认出则交 RAG 做接地讲解+指路（事实由知识库约束，压幻觉）
        location = None
```

```
    if req.ground:
      location = locate_landmark(
        image_b64=req.image_b64,
        scenic_name=cfg["name"],
        image_format=req.image_format,
      )
    if location:
      # 复用 97.7% 准确的 RAG 链路（自动走 navigation 意图 + group_type 人格）
      nav_query = (
        f"游客正站在{location}。请先用一句话确认所在位置，"
        f"再生动讲解{location}的文化背景与看点，"
        f"最后推荐接下来最值得去的1~2个邻近景点，并给出步行方向指引"
        f"（用相对方向，不要编造精确米数）。"
      )
      res = rag_answer(
        query=nav_query,
        scenic_id=req.scenic_id,
        session_id=req.session_id,
        lang=req.lang,
        group_type=req.group_type,
      )
      reply, emotion = res["reply"], res["emotion"]
    else:
      # 未识别（或 ground=False）：单次 VL，礼貌引导/直接看图作答
      reply, emotion, location = answer_with_image(
        image_b64=req.image_b64,
        question=req.question,
        scenic_name=cfg["name"],
        image_format=req.image_format,
      )
  except Exception as e:
    raise HTTPException(status_code=500, detail=f"多模态模型调用失败: {e}")
  loc_note = f"（定位：{location}）" if location else ""
  save_conversation(
    session_id=req.session_id,
    scenic_id=req.scenic_id,
    user_message=f"[拍照] {req.question}{loc_note}",
    ai_reply=reply,
    emotion=emotion,
    context_used=bool(location),
  )
  return ImageChatResponse(reply=reply, emotion=emotion, location=location)
==================== backend/api/analytics.py ====================
"""
数据分析接口，供管理后台数据大屏调用。
GET /analytics/dashboard     — 核心运营数据（总量、情绪分布、每日趋势）
GET /analytics/hot-topics    — 游客热门问题词云
GET /analytics/sentiment-trend — 满意度按天趋势
GET /analytics/conversations  — 历史对话列表（分页）
POST /analytics/report       — LLM 生成游客感受度报告（按需调用）
```

```
转写结果可直接喂给 /chat 或桥接层 /live/ask，构成"语音问答"链路第一环。
"""
import os
import tempfile
from fastapi import APIRouter, File, Form, HTTPException, UploadFile
from backend.asr.transcriber import transcribe
router = APIRouter(prefix="/asr", tags=["asr"])
@router.post("")
async def asr_endpoint(
    file: UploadFile = File(...),
    scenic_id: str = Form("lingshan"),
    language: str = Form("zh"),
):
    """
    上传音频文件（wav/mp3/m4a/webm…）→ 返回转写文字。
    scenic_id 用于注入对应景区术语热词，提升专名识别。
    """
    data = await file.read()
    if not data:
        raise HTTPException(status_code=400, detail="音频为空")
    suffix = os.path.splitext(file.filename or "audio.wav")[1] or ".wav"
    with tempfile.NamedTemporaryFile(suffix=suffix, delete=False) as tmp:
        tmp.write(data)
        path = tmp.name
    try:
        return transcribe(path, scenic_id, language)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"ASR 转写失败: {e}")
    finally:
        os.unlink(path)
==================== backend/analytics/crowd.py ====================
"""客流热度感知：基于真实游客行为数据（14 万行）+ 时段模型，给导游做「错峰/淡旺季」建议。
数据真实部分（来自 示范景区公开资料包/景点景区旅游数据行为分析数据.xlsx）：
 - 各景点访问量 → 热门度基线（灵山胜境/灵山大佛/拈花湾/西湖 均在数据中）
 - visit_date 月份分布 → 真实季节性（旺季 7-9 月）
 - satisfaction / stay_duration → 满意度、平均停留
模型部分（原始数据无小时维度，明确标注为估计）：
 - 时段拥挤曲线 hour_factor(now) → 模拟"今日此刻"的拥挤程度
当前拥挤度 = 热门度基线 × 当月季节因子 × 当前时段因子 → 分档（空闲/适中/较多/繁忙）。
导游据此给「错峰」建议，体现真实导游的「随机应变」职责。
"""
from __future__ import annotations
import datetime as dt
from functools import lru_cache
from pathlib import Path
import pandas as pd
_DATA = Path(__file__).resolve().parents[2] / "示范景区公开资料包" / "景点景区旅游数据行为分析数据.xlsx"
# scenic_id → 数据中的 attraction_name（一个景区可能对应多条）。黄山/故宫不在数据中。
SCENIC_ATTRACTIONS: dict[str, list[str]] = {
    "lingshan": ["灵山胜境", "灵山大佛"],
```

```
    "nihuawan": ["禅意小镇·拈花湾"],
    "xihu": ["西湖风景名胜区"],
}
# 景区典型时段拥挤曲线（相对系数 0~1）。原始数据无小时，这是经验模型估计。
_HOUR_CURVE = {
    6: 0.10, 7: 0.20, 8: 0.40, 9: 0.65, 10: 0.88, 11: 1.00, 12: 0.92,
    13: 0.88, 14: 0.98, 15: 0.90, 16: 0.72, 17: 0.52, 18: 0.34, 19: 0.20,
}
# 拥挤度 = 季节因子 × 时段因子（两者都有现实意义；不乘全局热门度，
# 因为本数据是区域级、灵山等占比小，乘了会被压成恒"空闲"）。热门度仅用于排序/展示。
_LEVELS = [(0.40, "空闲"), (0.80, "适中"), (1.30, "较多"), (99, "繁忙")]
def _hour_factor(hour: int) -> float:
    if hour in _HOUR_CURVE:
        return _HOUR_CURVE[hour]
    return 0.10 if hour < 6 or hour > 19 else 0.5
def _level(index: float) -> str:
    for thr, name in _LEVELS:
        if index < thr:
            return name
    return "繁忙"
@lru_cache(maxsize=1)
def _stats() -> dict:
    """读一次 xlsx，缓存：每景点的访问量/满意度/停留/月度分布。"""
    # 只读需要的列，明显加快 14 万行 xlsx 的冷加载（openpyxl 逐格解析很慢）
    df = pd.read_excel(
        _DATA,
        usecols=["attraction_name", "satisfaction", "stay_duration", "visit_date"],
    )
    g = df.groupby("attraction_name")
    pop = g.size()
    pop_norm = (pop / pop.max()).round(3)
    sat = g["satisfaction"].mean().round(2)
    stay = g["stay_duration"].mean().round(1)
    # 每景点的月度访问量（用于真实季节因子）
    monthly = (
        df.assign(m=df["visit_date"].dt.month)
        .groupby(["attraction_name", "m"]).size()
    )
    return {
        "pop": pop.to_dict(),
        "pop_norm": pop_norm.to_dict(),
        "sat": sat.to_dict(),
        "stay": stay.to_dict(),
        "monthly": monthly,
    }
def _month_factor(attraction: str, month: int) -> float:
    """当月访问量相对该景点月均的倍数（真实季节性）。>1 旺季，<1 淡季。"""
    st = _stats()
    try:
        ms = st["monthly"].loc[attraction]
```

```
    except KeyError:
        return 1.0
    mean = ms.mean()
    if mean <= 0:
        return 1.0
    return round(float(ms.get(month, mean)) / float(mean), 2)
def attraction_crowd(attraction: str, when: dt.datetime | None = None) -> dict | None:
    """单个景点的当前拥挤度（含真实热门度/满意度 + 模型化的时段估计）。"""
    st = _stats()
    if attraction not in st["pop_norm"]:
        return None
    when = when or dt.datetime.now()
    mf = _month_factor(attraction, when.month)
    hf = _hour_factor(when.hour)
    index = round(mf * hf, 3)                    # 季节(真实) × 时段(模型)
    return {
        "attraction": attraction,
        "crowd_index": index,
        "crowd_level": _level(index),
        "popularity": st["pop_norm"][attraction],  # 真实热门度 0~1（仅排序/展示）
        "satisfaction": st["sat"].get(attraction),  # 真实平均满意度
        "avg_stay_h": st["stay"].get(attraction),  # 真实平均停留(小时)
        "month_factor": mf,                      # 真实季节因子
        "hour_factor": hf,                       # 模型时段因子
    }
def scenic_crowd(scenic_id: str, when: dt.datetime | None = None) -> dict:
    """某景区的客流概览：各（数据中可对应的）景点拥挤度 + 季节/错峰提示。"""
    when = when or dt.datetime.now()
    attrs = SCENIC_ATTRACTIONS.get(scenic_id, [])
    items = [c for a in attrs if (c := attraction_crowd(a, when))]
    items.sort(key=lambda x: x["crowd_index"], reverse=True)
    # 旺季提示（真实月度）
    in_peak = any(it["month_factor"] >= 1.1 for it in items)
    off_peak = items and all(it["month_factor"] <= 0.9 for it in items)
    season = "旅游旺季" if in_peak else ("旅游淡季" if off_peak else "平季")
    # 今日错峰时段（模型）：曲线低谷
    quiet_hours = "清晨 8 点前或傍晚 17 点后"
    return {
        "scenic_id": scenic_id,
        "as_of": when.strftime("%Y-%m-%d %H:%M"),
        "has_data": bool(items),
        "season": season,
        "quiet_hours": quiet_hours,
        "attractions": items,
        "note": "热门度/满意度/季节性来自真实游客行为数据；当前时段拥挤度为模型估计（原始数据无小时维度）。",
    }
def crowd_brief(scenic_id: str, when: dt.datetime | None = None) -> str:
    """给导游 prompt 注入的一句话客流上下文。无数据时返回空串。"""
    c = scenic_crowd(scenic_id, when)
    if not c["has_data"]:
```

```
        return ""
    top = c["attractions"][0]
    parts = [f"【实时客流参考】当前{c['as_of']}，属{c['season']}。"]
    levels = "；".join(f"{it['attraction']}{it['crowd_level']}" for it in c["attractions"])
    parts.append(f"客流：{levels}。")
    if top["crowd_level"] in ("较多", "繁忙"):
        parts.append(f"{top['attraction']}此刻人较多，可建议游客错峰（{c['quiet_hours']}人少），或先逛相对清静
    处。")
    return "".join(parts)
==================== backend/analytics/env_context.py ====================
"""时间 + 天气感知：给导游"随机应变"的环境上下文（午后日晒→室内、下雨→带伞、傍晚→日落）。
- 时间：本地当前时刻 → 时段（清晨/上午/中午/下午/傍晚/夜晚），永远可用、零依赖。
- 天气：wttr.in 免费无 key 查询当地实时天气，带 30 分钟缓存 + 超时 + 优雅降级
      （取不到天气时只用时间，不影响主流程）。
导游据此自然给"避暑/带伞/防晒/看日落"等建议，体现真实导游对天气的适应职责。
"""
from __future__ import annotations
import datetime as dt
import time
import httpx
# scenic_id → 天气查询城市
SCENIC_CITY = {
    "lingshan": "无锡",
    "nihuawan": "无锡",
    "xihu": "杭州",
    "huangshan": "黄山",
    "gugong": "北京",
    "zhangjiajie": "张家界",
    "bingmayong": "西安",
    "mogaoku": "敦煌",
}
# wttr.in 英文天气描述 → 中文（覆盖常见项；未命中则保留原文）
_COND_CN = {
    "Sunny": "晴", "Clear": "晴", "Partly cloudy": "多云", "Cloudy": "阴",
    "Overcast": "阴", "Mist": "薄雾", "Fog": "雾", "Patchy rain nearby": "局部有雨",
    "Patchy rain possible": "可能有雨", "Light rain": "小雨", "Moderate rain": "中雨",
    "Heavy rain": "大雨", "Light drizzle": "毛毛雨", "Thundery outbreaks possible": "可能有雷阵雨",
    "Light snow": "小雪", "Moderate snow": "中雪", "Heavy snow": "大雪", "Patchy snow nearby": "局部有雪",
    "Haze": "雾霾", "Smoky haze": "雾霾", "Smoke": "烟霾", "Patchy light rain": "局部小雨",
    "Light rain shower": "阵雨", "Moderate or heavy rain shower": "大阵雨", "Blowing dust": "浮尘",
}
_CACHE: dict[str, tuple[float, str | None]] = {}  # city -> (ts, "中文天气 温度°C" 或 None)
_TTL = 1800  # 天气缓存 30 分钟
def _period(hour: int) -> str:
    if hour < 6:  return "凌晨"
    if hour < 9:  return "清晨"
    if hour < 11: return "上午"
    if hour < 13: return "中午"
    if hour < 17: return "下午"
    if hour < 19: return "傍晚"
```

```
    return "夜晚"
def _fetch_weather(city: str) -> str | None:
    """查 wttr.in，返回"多云 25°C"样式；失败返回 None。带 30 分钟缓存。"""
    now = time.time()
    hit = _CACHE.get(city)
    if hit and now - hit[0] < _TTL:
        return hit[1]
    result = None
    try:
        # trust_env=False：绕开环境里的 socks 代理（httpx 不支持 socks scheme），wttr.in 公网直连
        with httpx.Client(trust_env=False, timeout=5, follow_redirects=True) as c:
            r = c.get(f"https://wttr.in/{city}", params={"format": "%C|%t"})
        if r.status_code == 200 and "|" in r.text:
            cond_en, temp = (r.text.strip().split("|", 1) + [""])[:2]
            cond = _COND_CN.get(cond_en.strip(), cond_en.strip())
            temp = temp.strip().replace("+", "").replace("°C", "℃")
            result = f"{cond} {temp}".strip()
    except Exception:
        result = None
    _CACHE[city] = (now, result)
    return result
def env_brief(scenic_id: str, when: dt.datetime | None = None) -> str:
    """给导游 prompt 注入的一句话环境上下文（时间永远有，天气尽力而为）。"""
    when = when or dt.datetime.now()
    period = _period(when.hour)
    parts = [f"【实时环境】现在是 {when.month} 月 {when.day} 日{period}"]
    city = SCENIC_CITY.get(scenic_id)
    weather = _fetch_weather(city) if city else None
    if weather:
        parts.append(f"，{city}天气{weather}")
    parts.append("。请据此自然给出贴心建议（如午后日晒强先逛室内、下雨提醒带伞或走廊道、")
    parts.append("傍晚推荐日落机位、天热提醒补水），不要生硬罗列，也不要提及'系统/数据'。")
    return "".join(parts)
==================== backend/asr/transcriber.py ====================
"""
本地 ASR：基于 faster-whisper(CTranslate2)的中文景区语音识别模块（自研封装）。
相对裸用 faster-whisper 的改造点（答辩可讲的技术增量）：
 1. 景区术语增强：注入各景区专有名词作热词(hotwords)，显著提升
    "九龙灌浴""阿育王柱"等生僻景点名的识别率——whisper 原生对这类专名易错。
 2. VAD 静音过滤：切除静音段，加速推理并抑制噪声误识别。
 3. CPU int8 运行：0 显存占用，不与本机 MuseTalk 抢 4080 的显存。
 4. 中文后处理：去首尾空白、规整。
懒加载：import 本模块不触发模型加载；首次 transcribe 才加载模型
（∴ 无 GPU/未装 faster-whisper 的 ECS 即便注册了路由也不会在启动时崩）。
"""
import os
import re
from functools import lru_cache
# 去掉中文字符之间的空格（small 模型偶尔把"九龙灌浴"切成"九龙 灌浴"）
_CJK_SPACE = re.compile(r"(?<=[一-▯])\s+(?=[一-▯])")
```

```
# 各景区专有名词热词表（提升 ASR 对生僻景点名的识别；新增景区时在此补充）
SCENIC_HOTWORDS: dict[str, str] = {
  "lingshan":  "灵山胜境 灵山大佛 九龙灌浴 阿育王柱 五印坛城 梵宫 降魔成道 百子戏弥勒 祥符禅寺 天下第一掌 五
  智门",
  "nihuawan":  "拈花湾 禅意小镇 香月花街 五灯湖 拈花塔 半山衔日 鹿鸣谷 波罗蜜广场",
  "xihu":     "西湖 断桥残雪 雷峰塔 三潭印月 苏堤春晓 曲院风荷 花港观鱼 平湖秋月 灵隐寺 六和塔",
  "huangshan": "黄山 迎客松 光明顶 莲花峰 天都峰 飞来石 始信峰 西海大峡谷 鳌鱼峰 一线天",
  "gugong":    "故宫 紫禁城 太和殿 乾清宫 养心殿 御花园 角楼 午门 神武门 保和殿",
}
@lru_cache(maxsize=1)
def get_model():
  """懒加载 WhisperModel，进程内单例。首次调用需几秒加载（或下载）模型。"""
  os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
  from faster_whisper import WhisperModel
  from backend.core.config import settings
  return WhisperModel(
    settings.asr_model,
    device=settings.asr_device,
    compute_type=settings.asr_compute,
    cpu_threads=settings.asr_cpu_threads,
  )
def transcribe(audio_path: str, scenic_id: str = "lingshan",
         language: str = "zh") -> dict:
  """
  转写音频文件为文字。
  language: "zh" 强制中文；"auto" 自动检测。
  返回: { text, language, duration, hotwords_used }
  """
  from backend.core.config import settings
  model = get_model()
  hotwords = SCENIC_HOTWORDS.get(scenic_id, "")
  segments, info = model.transcribe(
    audio_path,
    language=None if language == "auto" else language,
    hotwords=hotwords or None,      # 景区术语增强
    vad_filter=True,              # VAD 静音过滤
    vad_parameters=dict(min_silence_duration_ms=500),
    beam_size=settings.asr_beam_size,
  )
  text = "".join(seg.text for seg in segments).strip()
  text = _CJK_SPACE.sub("", text)     # 修复中文专名被切开的空格
  return {
    "text": text,
    "language": info.language,
    "duration": round(info.duration, 2),
    "hotwords_used": bool(hotwords),
  }
==================== backend/db/database.py ====================
"""
SQLite 会话存储。
```

```
表结构：
 conversations - 每轮问答一行，存 session_id / scenic_id / 用户问 / AI答 / 情绪 / 时间戳（服务端自动埋点）
 events       - 客户端主动上报的行为事件（画像/功能使用/到访/满意度等服务端看不到的信号）
 scenic_theme  - 景区主题色覆盖（管理端 PUT 覆盖默认；客户端切景区时套用）
"""
import json
import sqlite3
from contextlib import contextmanager
from pathlib import Path
from backend.core.config import settings
DB_PATH: Path = settings.vector_store_dir.parent / "conversations.db"
def init_db() -> None:
  with sqlite3.connect(DB_PATH) as conn:
    conn.execute("""
      CREATE TABLE IF NOT EXISTS conversations (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        session_id   TEXT    NOT NULL,
        scenic_id    TEXT    NOT NULL DEFAULT 'lingshan',
        user_message TEXT    NOT NULL,
        ai_reply     TEXT    NOT NULL,
        emotion      TEXT    NOT NULL DEFAULT 'neutral',
        context_used INTEGER NOT NULL DEFAULT 1,
        created_at   TEXT    NOT NULL
                DEFAULT (strftime('%Y-%m-%d %H:%M:%S', 'now', 'localtime'))
      )
    """)
    conn.execute("CREATE INDEX IF NOT EXISTS idx_scenic_time ON conversations(scenic_id, created_at)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_session     ON conversations(session_id)")
    conn.execute("""
      CREATE TABLE IF NOT EXISTS events (
        id         INTEGER PRIMARY KEY AUTOINCREMENT,
        session_id  TEXT NOT NULL,
        scenic_id   TEXT NOT NULL DEFAULT 'lingshan',
        type       TEXT NOT NULL,
        group_type  TEXT,
        lang       TEXT,
        payload    TEXT,
        created_at  TEXT NOT NULL
              DEFAULT (strftime('%Y-%m-%d %H:%M:%S', 'now', 'localtime'))
      )
    """)
    conn.execute("CREATE INDEX IF NOT EXISTS idx_evt_type_time ON events(type, created_at)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_evt_scenic    ON events(scenic_id, created_at)")
    conn.execute("CREATE INDEX IF NOT EXISTS idx_evt_session   ON events(session_id)")
    conn.execute("""
      CREATE TABLE IF NOT EXISTS scenic_theme (
        scenic_id    TEXT PRIMARY KEY,
        name         TEXT,
        primary_color TEXT,
        sweet_color   TEXT,
```

```
            updated_at    TEXT NOT NULL
                    DEFAULT (strftime('%Y-%m-%d %H:%M:%S', 'now', 'localtime'))
        )
    """)
    conn.execute("""
        CREATE TABLE IF NOT EXISTS digital_human_config (
            scenic_id    TEXT PRIMARY KEY,
            avatar_id    TEXT NOT NULL DEFAULT 'female_elegant',
            avatar_name  TEXT NOT NULL DEFAULT '素雅女性',
            voice_id     TEXT NOT NULL DEFAULT 'voice_gentle_f',
            voice_name   TEXT NOT NULL DEFAULT '温柔女声',
            updated_at   TEXT NOT NULL
                    DEFAULT (strftime('%Y-%m-%d %H:%M:%S', 'now', 'localtime'))
        )
    """)
    conn.commit()
@contextmanager
def get_db():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()
def save_conversation(
    session_id: str,
    scenic_id: str,
    user_message: str,
    ai_reply: str,
    emotion: str,
    context_used: bool,
) -> None:
    with get_db() as conn:
        conn.execute(
            """INSERT INTO conversations
              (session_id, scenic_id, user_message, ai_reply, emotion, context_used)
              VALUES (?, ?, ?, ?, ?, ?)""",
            (session_id, scenic_id, user_message, ai_reply, emotion, int(context_used)),
        )
def save_event(
    session_id: str,
    scenic_id: str,
    type: str,
    group_type: str | None = None,
    lang: str | None = None,
    payload: dict | None = None,
) -> None:
    """落库一条客户端上报事件。payload 存 JSON 字符串，容错（坏 payload 不抛）。"""
    try:
```

```
    payload_str = json.dumps(payload, ensure_ascii=False) if payload else None
  except (TypeError, ValueError):
    payload_str = None
  with get_db() as conn:
    conn.execute(
      """INSERT INTO events (session_id, scenic_id, type, group_type, lang, payload)
        VALUES (?, ?, ?, ?, ?, ?)""",
      (session_id, scenic_id, type, group_type, lang, payload_str),
    )
def load_theme_override(scenic_id: str) -> dict | None:
  """读某景区的主题覆盖；无则 None。只返回非空字段。"""
  with get_db() as conn:
    row = conn.execute(
      "SELECT name, primary_color, sweet_color FROM scenic_theme WHERE scenic_id=?",
      (scenic_id,),
    ).fetchone()
  if not row:
    return None
  out = {}
  if row["name"]:        out["name"] = row["name"]
  if row["primary_color"]: out["primary"] = row["primary_color"]
  if row["sweet_color"]:   out["sweet"] = row["sweet_color"]
  return out or None
def save_theme(scenic_id: str, name: str | None, primary: str | None, sweet: str | None) -> None:
  """管理端覆盖某景区主题（部分字段：保留已有、只更新传入的非空项）。"""
  with get_db() as conn:
    conn.execute(
      """INSERT INTO scenic_theme (scenic_id, name, primary_color, sweet_color, updated_at)
        VALUES (?, ?, ?, ?, strftime('%Y-%m-%d %H:%M:%S','now','localtime'))
        ON CONFLICT(scenic_id) DO UPDATE SET
          name        = COALESCE(excluded.name, scenic_theme.name),
          primary_color = COALESCE(excluded.primary_color, scenic_theme.primary_color),
          sweet_color   = COALESCE(excluded.sweet_color, scenic_theme.sweet_color),
          updated_at    = excluded.updated_at""",
      (scenic_id, name, primary, sweet),
    )
==================== backend/core/config.py ====================
from pydantic_settings import BaseSettings
from pathlib import Path
BASE_DIR = Path(__file__).resolve().parent.parent.parent
class Settings(BaseSettings):
  # DashScope (通义千问)
  dashscope_api_key: str = ""
  # qa（事实问答）用模型。qwen-plus 比 qwen-max 生成快 ~2 倍（48 vs 20 字/s），
  # 流式首句从 ~3s 降到 ~1.2s，是压住语音端到端 <5s 的关键；44 题准确率 100%
  # （keyword 自测），事实由 RAG context 约束，质量足够。生成性意图另走 qwen-turbo。
  llm_model: str = "qwen-plus"
  vl_model: str = "qwen-vl-max"
  # 部署模式：local（本地GPU）或 cloud（阿里云，用DashScope Embedding）
  deploy_mode: str = "local"
```

```
    # 本地模型（deploy_mode=local 时使用）
    embedding_model: str = "BAAI/bge-m3"
    reranker_model: str = "BAAI/bge-reranker-v2-m3"
    # ASR（本地 faster-whisper；CPU 运行不抢数字人显存）
    asr_model: str = "small"      # 实测 small CPU ~1.1s 达标；medium 在本机 RTF≈2 太慢
    asr_device: str = "cpu"       # cpu：0 显存占用，把 4080 整块显存留给数字人(MuseTalk)
    asr_compute: str = "int8"     # int8：CPU 上最快
    asr_cpu_threads: int = 4      # 实测本机 4 线程最快（大小核架构，过多线程反增调度开销）
    asr_beam_size: int = 1        # 贪心解码；短问答足够，比 beam=5 快约 30%
    # RAG 参数
    chunk_size: int = 512
    chunk_overlap: int = 128
    dense_top_k: int = 10
    sparse_top_k: int = 10
    rerank_top_k: int = 4
    # EA-RAG 自研检索算法（Entity-Anchored Adaptive RAG）三机制开关
    # 全部可经环境变量覆盖，供消融实验逐项开关
    ea_entity_anchor: bool = True     # ① 实体锚定：query 命中景点名 → 该景点 chunk 加权
    ea_adaptive_fusion: bool = True   # ② 自适应融合：按 query 特征动态调 dense/sparse 权重的加权 RRF
    ea_confidence_gate: bool = True   # ③ 置信度门控：reranker top1 低于阈值 → 判定知识库无此信息
    ea_gate_threshold: float = 0.05   # 门控阈值（bge-reranker normalize 后相关>0.9，无关<0.01）
    # 路径
    vector_store_dir: Path = BASE_DIR / "vector_store"
    knowledge_base_dir: Path = BASE_DIR / "knowledge_base"
    class Config:
        env_file = BASE_DIR / ".env"
        env_file_encoding = "utf-8"
        extra = "ignore"
settings = Settings()
==================== bridge/server.py ====================
"""
RAG → 数字人 桥接层（本地运行，串联问答与口型驱动）
部署位置：本地 4080 机器，与 LiveTalking 同机（内部 localhost 调用，零公网暴露）。
职责（把两条孤立的服务接成一条链）：
  游客提问 → 本桥接 /live/ask
    1) 调 RAG_BASE/chat       拿 reply / emotion / intent
    2) 读 digital-human/config 把占位 avatar_id/voice_id 翻译成
       「真实的 LiveTalking avatar」与「真实的 edge-TTS 音色」
    3) 调 LIVETALKING_BASE/human(echo) 让数字人用该音色把 reply 说出来
  形象切换：avatar 是「连接级」参数（在 LiveTalking /offer 建连时指定），
  所以前端连接前先调 /live/avatar 拿到该景区当前 avatar_id，再传给 /offer。
启动：
  cd /home/jiale/Desktop/Digital_Person
  .venv/bin/uvicorn bridge.server:app --host 0.0.0.0 --port 9000
可用环境变量覆盖后端地址（演示时 RAG 可指向本地 GPU 实例以求最低延迟，
也可指向 ECS 公网 http://116.62.136.211:8000）：
  RAG_BASE=http://localhost:8000  LIVETALKING_BASE=http://localhost:8010
"""
import json
import os
```

```
import pathlib
import httpx
from fastapi import FastAPI, HTTPException, Request
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse, Response, FileResponse
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel
RAG_BASE = os.environ.get("RAG_BASE", "http://localhost:8000").rstrip("/")
LIVETALKING_BASE = os.environ.get("LIVETALKING_BASE", "http://localhost:8010").rstrip("/")
# SRS 的 HTTP-FLV 直播流地址（RTMP 模式下 LiveTalking 推到 SRS，浏览器从这里拉）
SRS_FLV = os.environ.get("SRS_FLV", "http://localhost:8080/live/livestream.flv").rstrip("/")
# SRS 的 HLS 目录（供 iOS H5 经桥接同源反代，绕开 mixed-content + 跨源）
SRS_HLS_BASE = os.environ.get("SRS_HLS_BASE", "http://localhost:8080/live").rstrip("/")
# React 商业级前端构建产物（webapp/dist），桥接当统一 https 入口在末尾 mount
WEBAPP_DIST = os.environ.get(
    "WEBAPP_DIST",
    str(pathlib.Path(__file__).resolve().parent.parent / "webapp" / "dist"),
)
# 旧版纯手写 H5（保留回退，挂在 /legacy）
H5_MOBILE_PATH = os.environ.get(
    "H5_MOBILE_PATH",
    str(pathlib.Path(__file__).resolve().parent.parent / "talk_mobile.html"),
)
# 安卓 FLV 播放器（mpegts.js）：随 LiveTalking 仓库自带，桥接同源转发给安卓浏览器/WebView
MPEGTS_JS_PATH = os.environ.get(
    "MPEGTS_JS_PATH",
    os.path.expanduser("~/Desktop/LiveTalking/web/mpegts-1.7.3.min.js"),
)
# ── 占位 ID → 真实资源的映射 ──────────────────────────────────────────
# digital-human/config 里存的是业务占位 ID（female_elegant / voice_gentle_f…），
# 这里把它们翻译成下游真正认识的资源名。生成新形象/接入新音色后只改这两张表。
# voice_id → 百炼 Qwen-TTS 音色名（LiveTalking --tts qwentts 读 tts.ref_file）
# Qwen3-TTS-Flash-Realtime：云端实时合成、0 本地 GPU、用现有 DASHSCOPE_API_KEY，
# 自然度远胜 edge-TTS。音色按消息可切（qwentts 检测到 ref_file 变化即重连换音色）。
VOICE_MAP = {
    "voice_gentle_f": "Serena",  # 知性温柔女声（默认，慧昭人设——Cherry 偏嫩已弃用）
    "voice_lively_f": "Cherry",  # 活泼女声（保留甜美系，按形象可切）
    "voice_calm_m":   "Ethan",   # 沉稳男声
    "voice_bright_m": "Ethan",   # 明朗男声
}
DEFAULT_VOICE = "Serena"
# avatar_id → 真实 LiveTalking avatar 目录名（data/avatars/<id>）
# 古风女导游 guofeng_female 由通义万相生成（scripts/gen_avatar_asset.py）→ genavatar 烘成。
AVATAR_MAP = {
    "female_elegant": "sweet_bob",  # 主形象：美图精修的甜美波波头(眼袋淡/鼻孔小/无穿模)
    "female_young":   "sweet_bob",
    "male_mature":    "musetalk_avatar1",  # TODO: 生成男导游后改这里
    "male_young":     "musetalk_avatar1",
}
DEFAULT_AVATAR = "sweet_bob"
```

```
app = FastAPI(title="数字人桥接层", version="0.1.0")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)
class AskIn(BaseModel):
    sessionid: str              # LiveTalking WebRTC 会话 ID（/offer 返回的那个）
    message: str
    session_id: str = "anonymous"  # RAG 业务会话 ID（用于历史/分析，与上面不是一回事）
    scenic_id: str = "lingshan"
    group_type: str = "adult"     # adult / family_kids / senior / couple
    lang: str = "auto"
    fast: bool = False
    interrupt: bool = True        # 新提问默认打断数字人上一句未播完的话
async def _resolve_config(client: httpx.AsyncClient, scenic_id: str) -> tuple[str, str]:
    """读 digital-human/config，翻译成 (真实 avatar, 真实 voice)。任何失败回落默认。"""
    try:
        r = await client.get(
            f"{RAG_BASE}/digital-human/config",
            params={"scenic_id": scenic_id},
            timeout=5,
        )
        cfg = r.json()
        avatar = AVATAR_MAP.get(cfg.get("avatar_id"), DEFAULT_AVATAR)
        voice = VOICE_MAP.get(cfg.get("voice_id"), DEFAULT_VOICE)
        return avatar, voice
    except Exception:
        return DEFAULT_AVATAR, DEFAULT_VOICE
@app.post("/live/ask")
async def live_ask(body: AskIn):
    """游客问一句 → 订阅 RAG 句子级流式 → 每出一句立即驱动数字人开口。
    关键时序（压住 <5s 端到端延迟的核心）：
      RAG 第 1 句生成完(~0.8s) → 立刻送 LiveTalking 说出来 → 数字人即刻开口，
      与此同时 RAG 继续生成后面的句子并排队播放，而不是等整段答案。
    """
    # trust_env=False：桥接只调 localhost/内网/ECS，绕开环境里的 SOCKS 代理
    # read 超时给到 60s，容纳 LLM 整段流式生成的时长
    timeout = httpx.Timeout(connect=5.0, read=60.0, write=10.0, pool=5.0)
    async with httpx.AsyncClient(trust_env=False, timeout=timeout) as client:
        # 1) 流式开始前先确定音色（第一句就要用）
        _, voice = await _resolve_config(client, body.scenic_id)
        intent = "qa"
        emotion = "neutral"
        sentences: list[str] = []
        drive_failures = 0
        first = True
        async def _drive(text: str, interrupt: bool):
            """把一句话送数字人说出来。单句失败不中断整条流，只计数。"""
```

```
        nonlocal drive_failures
        try:
            await client.post(
                f"{LIVETALKING_BASE}/human",
                json={
                    "sessionid": body.sessionid,
                    "text": text,
                    "type": "echo",
                    "interrupt": interrupt,
                    # ref_file=Qwen音色名(qwentts用); scenic_id=景区(ttsbackend用→队友CosyVoice景区专属音
色);
                    # emotion 透传→数字人切表情底座
                    "tts": {"ref_file": voice, "emotion": emotion, "scenic_id": body.scenic_id},
                },
                timeout=10,
            )
        except Exception:
            drive_failures += 1
    # 2) 订阅 RAG SSE 流，逐句驱动数字人
    try:
        async with client.stream(
            "POST",
            f"{RAG_BASE}/chat",
            json={
                "session_id": body.session_id,
                "message": body.message,
                "scenic_id": body.scenic_id,
                "group_type": body.group_type,
                "lang": body.lang,
                "fast": body.fast,
                "stream": True,
            },
        ) as resp:
            resp.raise_for_status()
            async for line in resp.aiter_lines():
                if not line.startswith("data:"):
                    continue
                try:
                    evt = json.loads(line[5:].strip())
                except json.JSONDecodeError:
                    continue
                etype = evt.get("type")
                if etype == "intent":
                    intent = evt.get("value", intent)
                elif etype == "emotion":
                    # emotion 事件在第一个 sentence 之前下发，第一句即可带上正确情绪
                    emotion = evt.get("value", emotion)
                elif etype == "sentence":
                    text = (evt.get("text") or "").strip()
                    if not text:
```

```
                continue
            sentences.append(text)
            # 第一句打断上一轮残留，后续句排队顺序播放
            await _drive(text, interrupt=(body.interrupt and first))
            first = False
        # end 事件无需特殊处理
    except Exception as e:
        raise HTTPException(
            status_code=502,
            detail=f"RAG 流式调用失败: {type(e).__name__}: {e}",
        )
    if not sentences:
        raise HTTPException(status_code=502, detail="RAG 未产出任何句子")
    if drive_failures == len(sentences):
        raise HTTPException(
            status_code=502,
            detail=f"LiveTalking 驱动失败：{len(sentences)} 句均未送达",
        )
    return {
        "ok": drive_failures == 0,
        "reply": "".join(sentences),
        "emotion": emotion,
        "intent": intent,
        "voice": voice,
        "sentence_count": len(sentences),
        "drive_failures": drive_failures,
    }
class SayIn(BaseModel):
    sessionid: str              # LiveTalking WebRTC/RTMP 会话 ID
    text: str                   # 直接要数字人说出来的文字（如拍照定位讲解的结果）
    scenic_id: str = "lingshan"   # 决定音色
    interrupt: bool = True
    emotion: str = "neutral"      # 面部情绪：驱动数字人切换表情底座（neutral/happy/concerned…）
@app.post("/live/say")
async def live_say(body: SayIn):
    """让数字人直接说出一段给定文字（不走 RAG）。
    用于「拍照视觉定位」闭环：游客拍照 → /chat/image 出定位+讲解+指路文字 →
    本端点把这段文字按景区音色送 LiveTalking /human，让数字人开口说。
    """
    text = (body.text or "").strip()
    if not text:
        raise HTTPException(status_code=400, detail="text 不能为空")
    async with httpx.AsyncClient(trust_env=False, timeout=httpx.Timeout(connect=5.0, read=30.0, write=10.0, po
    ol=5.0)) as client:
        _, voice = await _resolve_config(client, body.scenic_id)
        try:
            await client.post(
                f"{LIVETALKING_BASE}/human",
                json={
                    "sessionid": body.sessionid,
```

```
                "text": text,
                "type": "echo",
                "interrupt": body.interrupt,
                "tts": {"ref_file": voice, "emotion": body.emotion, "scenic_id": body.scenic_id},
            },
            timeout=15,
        )
    except Exception as e:
        raise HTTPException(status_code=502, detail=f"驱动数字人失败: {type(e).__name__}: {e}")
  return {"ok": True, "voice": voice, "chars": len(text)}
@app.get("/stream.flv")
async def stream_flv():
  """反代 SRS 的 HTTP-FLV 直播流。
  为什么要反代：SRS 的 FLV 响应不带 Access-Control-Allow-Origin，
  而交互页是从 :8000 (或别的源) 打开的，浏览器跨源拉 :8080 的流会被拦 → 画框转圈/黑屏。
  桥接层(:9000)有 CORS allow *，前端改拉 :9000/stream.flv 即同源放行，绕开 SRS 无 CORS。
  用 StreamingResponse 边收边吐，不缓冲整段（直播流是无限长的）。
  """
  client = httpx.AsyncClient(trust_env=False, timeout=httpx.Timeout(connect=5.0, read=None, write=10.0, pool
  =5.0))
  req = client.build_request("GET", SRS_FLV)
  upstream = await client.send(req, stream=True)
  if upstream.status_code != 200:
    await upstream.aclose()
    await client.aclose()
    raise HTTPException(status_code=502, detail=f"SRS 流不可用: HTTP {upstream.status_code}（确认 LiveTalk
  ing 正在推流、SRS 在跑）")
  async def gen():
    try:
      async for chunk in upstream.aiter_bytes():
        yield chunk
    finally:
      await upstream.aclose()
      await client.aclose()
  return StreamingResponse(gen(), media_type="video/x-flv")
# ── iOS H5 统一入口：桥接(:9000) 当唯一 https 源，反代 RAG + SRS-HLS ──────────
# 背景：个人号小程序 <video> 播不了直播 HLS、又禁 live-player；改走 iPhone Safari
# （原生支持 HLS）。但 Safari 上 https 页面不能混入 http 子资源（mixed-content），
# 且多端口会跨源。故让桥接做唯一入口：H5 全部相对路径，桥接反代到各后端，
# 再用 cloudflared 把 :9000 暴露成可信 https 域名。
@app.get("/legacy")
async def serve_h5_legacy():
  """旧版纯手写 H5（回退）。商业级 React 版由文件末尾 StaticFiles 挂在 /。"""
  return FileResponse(H5_MOBILE_PATH, media_type="text/html")
@app.get("/vendor/mpegts.js")
async def serve_mpegts():
  """安卓浏览器/WebView 的 FLV 播放器，同源转发避免跨源/混合内容。"""
  return FileResponse(MPEGTS_JS_PATH, media_type="application/javascript")
@app.get("/hls/{path:path}")
async def proxy_hls(path: str):
```

```
    """反代 SRS 的 HLS（m3u8 + ts）。m3u8 里 ts 是相对路径，仍命中本路由。"""
    url = f"{SRS_HLS_BASE}/{path}"
    async with httpx.AsyncClient(trust_env=False,
                                 timeout=httpx.Timeout(connect=5.0, read=20.0, write=10.0, pool=5.0)) as clien
    t:
        r = await client.get(url)
    # m3u8 用标准 MIME；ts 用 video/mp2t。直播分片禁缓存。
    ct = r.headers.get("content-type") or (
        "application/vnd.apple.mpegurl" if path.endswith(".m3u8") else "video/mp2t"
    )
    return Response(content=r.content, status_code=r.status_code, media_type=ct,
                    headers={"Cache-Control": "no-cache"})
@app.api_route("/rag/{path:path}", methods=["GET", "POST"])
async def proxy_rag(path: str, request: Request):
    """反代 RAG 后端（/trip/greeting、/asr、/chat/image 等），转发方法/查询/体/头。"""
    url = f"{RAG_BASE}/{path}"
    body = await request.body()
    # 去掉 hop-by-hop 与会让上游错算长度的头
    headers = {k: v for k, v in request.headers.items()
               if k.lower() not in ("host", "content-length", "connection", "accept-encoding")}
    async with httpx.AsyncClient(trust_env=False,
                                 timeout=httpx.Timeout(connect=5.0, read=60.0, write=30.0, pool=5.0)) as clien
    t:
        r = await client.request(request.method, url, params=dict(request.query_params),
                                 content=body, headers=headers)
    resp_headers = {k: v for k, v in r.headers.items()
                    if k.lower() not in ("content-length", "transfer-encoding", "connection", "content-encodin
    g")}
    return Response(content=r.content, status_code=r.status_code,
                    media_type=r.headers.get("content-type"), headers=resp_headers)
@app.get("/live/avatar")
async def live_avatar(scenic_id: str = "lingshan"):
    """前端建立 WebRTC 连接前调用：返回该景区当前应使用的真实 avatar_id 与音色。
    把返回的 avatar_id 作为 LiveTalking /offer 的 `avatar` 参数即可实现形象切换。"""
    async with httpx.AsyncClient(trust_env=False) as client:
        avatar, voice = await _resolve_config(client, scenic_id)
    return {"avatar_id": avatar, "voice": voice, "scenic_id": scenic_id}
@app.get("/live/health")
async def live_health():
    """探测 RAG 与 LiveTalking 是否都在线（联调时先打这个）。"""
    out = {
        "rag": False,
        "livetalking": False,
        "rag_base": RAG_BASE,
        "livetalking_base": LIVETALKING_BASE,
    }
    async with httpx.AsyncClient(trust_env=False) as client:
        try:
            r = await client.get(f"{RAG_BASE}/health", timeout=3)
            out["rag"] = r.status_code == 200
```

```
    except Exception:
        pass
    try:
        # LiveTalking 无 /health，用首页探活
        r = await client.get(f"{LIVETALKING_BASE}/index.html", timeout=3)
        out["livetalking"] = r.status_code == 200
    except Exception:
        pass
  return out
# ── React 商业级前端：StaticFiles 挂在 /（必须置于所有 API 路由之后，API 优先匹配）──
# html=True 让 / 发 index.html、/assets/* 发静态资源；SPA 单页直接命中 index。
# dist 不存在（未构建）则回退发旧版手写 H5，保证始终有可用页面。
if os.path.isdir(WEBAPP_DIST):
  app.mount("/", StaticFiles(directory=WEBAPP_DIST, html=True), name="webapp")
else:
  @app.get("/")
  async def _serve_legacy_root():
    return FileResponse(H5_MOBILE_PATH, media_type="text/html")
==================== tests/eval_kb.py ====================
"""
知识库自省 + 测试题自动派生 + 覆盖率计算。
设计目标（对应「面面俱到 + 覆盖率100%」的说服力诉求）：
 1. 从结构化数据集 docx **逐景点逐字段** 解析出全部知识点 → 这是覆盖率的分母。
 2. 为每个景点自动派生考题，关键词从该景点字段里抽取**确定性事实**
    （数字 + 书名号「」/引号"" 内的专名），保证评分可复现、评委可肉眼核对。
 3. 覆盖率 = 被至少一题考查的景点数 ÷ 总景点数 → 由构造保证 100%。
为什么自动派生而非手挑：手挑会被质疑「只挑好答的」。自动遍历每个景点每个
字段，无人为筛选，准确率数字才有说服力。
"""
import re
from pathlib import Path
from docx import Document
DATA_DIR = Path(__file__).resolve().parent.parent / "示范景区公开资料包"
# 结构化数据集文件 → (scenic_id, 表索引)。灵山 docx 内含两张表：表0灵山、表1拈花湾。
STRUCTURED_SOURCES = [
  ("lingshan", "灵山胜境 景点结构化数据集.docx",    0),
  ("nihuawan", "灵山胜境 景点结构化数据集.docx",    1),
  ("xihu",    "杭州西湖 景点结构化数据集.docx",    0),
  ("huangshan", "安徽黄山 景点结构化数据集.docx",    0),
  ("gugong",   "北京故宫博物院 景点结构化数据集.docx", 0),
]
SCENIC_ZH = {
  "lingshan": "灵山胜境", "nihuawan": "拈花湾", "xihu": "西湖",
  "huangshan": "黄山", "gugong": "故宫博物院",
}
def _match_field(headers, *needles):
  """按表头模糊匹配字段下标（两套 schema 字段名略有差异）。"""
  for i, h in enumerate(headers):
    if any(n in h for n in needles):
      return i
```

```
    return None
def _extract_numbers(text: str) -> list[str]:
    """抽取带计量意义的数字（含小数），作为可肉眼核对的确定性关键词。"""
    nums = re.findall(r"\d+(?:\.\d+)?", text or "")
    # 过滤掉过短且无意义的（如单独的 0），保留有信息量的
    out, seen = [], set()
    for n in nums:
        if n in seen:
            continue
        seen.add(n)
        if len(n) >= 2 or "." in n:  # 39.8 / 2797 / 88 等
            out.append(n)
    return out
def _extract_terms(text: str) -> list[str]:
    """抽取引号""与书名号《》「」内的专名，作为文化类题的确定性关键词。"""
    terms = re.findall(r"[“\"「《]([^”\"」》]{2,12})[”\"」》]", text or "")
    out, seen = [], set()
    for t in terms:
        t = t.strip()
        if t and t not in seen:
            seen.add(t)
            out.append(t)
    return out
def load_attractions() -> list[dict]:
    """解析全部结构化数据集 → 景点记录列表。"""
    records = []
    for scenic_id, fname, tidx in STRUCTURED_SOURCES:
        doc = Document(str(DATA_DIR / fname))
        table = doc.tables[tidx]
        headers = [c.text.strip() for c in table.rows[0].cells]
        f_name = _match_field(headers, "景点名称")
        f_loc  = _match_field(headers, "具体位置", "位置")
        f_par  = _match_field(headers, "建筑/景观参数", "参数")
        f_cul  = _match_field(headers, "文化内涵", "历史背景")
        f_high = _match_field(headers, "游玩亮点", "推荐活动")
        f_intro = _match_field(headers, "详细介绍")
        for row in table.rows[1:]:
            cells = [c.text.strip() for c in row.cells]
            if not any(cells) or f_name is None:
                continue
            name = cells[f_name]
            if not name:
                continue
            records.append({
                "scenic_id": scenic_id,
                "scenic_zh": SCENIC_ZH[scenic_id],
                "name": name,
                "location": cells[f_loc] if f_loc is not None else "",
                "params": cells[f_par] if f_par is not None else "",
                "culture": cells[f_cul] if f_cul is not None else "",
```

```
            "highlights": cells[f_high] if f_high is not None else "",
            "intro": cells[f_intro] if f_intro is not None else "",
        })
    return records
def build_coverage_questions() -> list[dict]:
    """
    为每个景点自动派生考题。优先级：
     - 有数字参数 → 参数题（关键词=数字，强信号、可肉眼核对）
     - 有引号专名 → 文化题（关键词=专名）
     - 至少保底一道「介绍」题，保证该景点被覆盖（关键词=景点名核心词）
    返回题目列表，每题含 dimension 字段用于「字段维度覆盖」统计。
    """
    questions = []
    for rec in load_attractions():
        name, sid = rec["name"], rec["scenic_id"]
        asked = False
        # 参数题（数字关键词）
        nums = _extract_numbers(rec["params"])
        if nums:
            questions.append({
                "scenic_id": sid, "attraction": name, "dimension": "参数",
                "q": f"{name}的建筑或景观参数（尺寸、规模等）是多少？",
                "keywords": nums[:3],
            })
            asked = True
        # 文化题（专名关键词）
        terms = _extract_terms(rec["culture"] + " " + rec["intro"])
        if terms:
            questions.append({
                "scenic_id": sid, "attraction": name, "dimension": "文化",
                "q": f"{name}有什么文化内涵或历史背景？",
                "keywords": terms[:3],
            })
            asked = True
        # 保底覆盖题：确保每个景点至少被考查一次
        if not asked:
            core = re.sub(r"[（(].*?[)）]", "", name).strip()
            questions.append({
                "scenic_id": sid, "attraction": name, "dimension": "概览",
                "q": f"请介绍一下{name}。",
                "keywords": [core[:6]] if len(core) >= 2 else [name],
            })
    return questions
# ═══════════════════════════════════════════════════════════════════════════
# Tier 2 精选难题：考查叙述性文档（票价/交通/季节/历史数字），结构化表里没有
# 这些来自《游览指南》docx，与自动覆盖题互补，提高维度广度。
# （沿用并精简自 test_accuracy.py 的人工校验题，关键词已核对）
# ═══════════════════════════════════════════════════════════════════════════
CURATED = {
    "lingshan": [
```

```
        ("灵山大佛有多高？", ["88", "米"]),
        ("灵山大佛建于哪一年？", ["1997", "1994"]),
        ("灵山大佛是什么材质建造的？", ["青铜", "铜"]),
        ("灵山胜境位于哪个城市？", ["无锡", "江苏"]),
        ("灵山胜境的门票价格大概是多少？", ["门票", "元", "210", "230"]),
        ("五印坛城来自哪个佛教传统？", ["藏传", "藏族"]),
    ],
    "xihu": [
        ("西湖在哪个城市，湖面面积有多大？", ["杭州", "6.39"]),
        ("苏堤是谁修建的，全长多少米？", ["苏轼", "苏东坡", "2797"]),
        ("西湖三潭印月的标志图案印在哪里？", ["一元", "硬币", "人民币"]),
        ("西湖何时被列入世界文化遗产？", ["2011"]),
        ("雷峰塔现在的塔有多高，是哪年重建的？", ["71", "2002"]),
    ],
    "huangshan": [
        ("黄山最高峰是哪座峰，海拔多少米？", ["莲花峰", "1864"]),
        ("黄山以什么四绝著称？", ["奇松", "怪石", "云海", "温泉"]),
        ("迎客松有多少年树龄？", ["1300"]),
        ("黄山是哪年被列入世界遗产的？", ["1990"]),
        ("黄山温泉的水温大约是多少度？", ["42"]),
    ],
    "gugong": [
        ("故宫建于哪一年，历时多久建成？", ["1406", "1420", "14年"]),
        ("太和殿建筑面积有多大？", ["2377"]),
        ("故宫一共有多少间房屋？", ["9000"]),
        ("故宫博物院现藏有多少件文物？", ["186万"]),
        ("故宫旺季和淡季的门票分别是多少钱？", ["60", "40"]),
    ],
    "nihuawan": [
        ("拈花湾与灵山胜境是什么关系？", ["禅意", "配套", "相邻", "灵山"]),
        ("拈花湾的开放时间是几点到几点？", ["21:30", "9:00", "21", "9"]),
    ],
}
def build_curated_questions() -> list[dict]:
    out = []
    for sid, items in CURATED.items():
        for q, kws in items:
            out.append({"scenic_id": sid, "attraction": "(精选)", "dimension": "实用/历史",
                        "q": q, "keywords": kws})
    return out
# ═══════════════════════════════════════════════════════════════════════════
# Tier 3 陷阱集：测「拒答 / 纠错」能力 —— 证明数字人不幻觉、可信赖。
#   kind=out_of_kb   知识库外问题 → 应兜底（不编）
#   kind=wrong_premise 错误前提 → 应纠正（给正确值或否定）
#   kind=cross_scenic  跨景区串问 → 不应给出他景区的具体答案
# 评分：命中 refuse_markers（兜底话术）或 correct_keywords（纠正事实）即算「应对正确」。
# ═══════════════════════════════════════════════════════════════════════════
REFUSE_MARKERS = ["工作人员", "现场", "公告", "无法", "没有相关", "暂无", "查阅",
                  "不太清楚", "建议您", "咨询", "未提及", "并非", "不是", "实际上",
                  "应为", "其实", "不在", "不属于", "并不", "没有", "无关", "不在本",
```

```
            "纠正", "误会", "其实是", "距离"]
TRAP_CASES = [
    # 知识库外
    {"scenic_id": "lingshan", "kind": "out_of_kb",
     "q": "灵山胜境景区里的肯德基几点关门？", "correct_keywords": []},
    {"scenic_id": "lingshan", "kind": "out_of_kb",
     "q": "灵山大佛的设计师生日是哪天？", "correct_keywords": []},
    {"scenic_id": "xihu", "kind": "out_of_kb",
     "q": "西湖边那家星巴克的WiFi密码是多少？", "correct_keywords": []},
    {"scenic_id": "gugong", "kind": "out_of_kb",
     "q": "故宫现任院长的手机号码是多少？", "correct_keywords": []},
    {"scenic_id": "huangshan", "kind": "out_of_kb",
     "q": "黄山明天下午三点整会不会下雨？", "correct_keywords": []},
    # 错误前提（应纠正）
    {"scenic_id": "lingshan", "kind": "wrong_premise",
     "q": "听说灵山大佛有120米高，是真的吗？", "correct_keywords": ["88"]},
    {"scenic_id": "lingshan", "kind": "wrong_premise",
     "q": "灵山大佛是不是用纯金打造的？", "correct_keywords": ["青铜", "铜"]},
    {"scenic_id": "xihu", "kind": "wrong_premise",
     "q": "苏堤是李白主持修建的对吧？", "correct_keywords": ["苏轼", "苏东坡"]},
    {"scenic_id": "xihu", "kind": "wrong_premise",
     "q": "西湖是不是要收200元门票才能进？", "correct_keywords": ["免费", "免"]},
    {"scenic_id": "huangshan", "kind": "wrong_premise",
     "q": "黄山最高峰是天都峰对不对？", "correct_keywords": ["莲花峰", "1864"]},
    {"scenic_id": "gugong", "kind": "wrong_premise",
     "q": "故宫是清朝康熙年间才开始建的吧？", "correct_keywords": ["1406", "明", "永乐"]},
    # 跨景区串问（在A景区知识库问B景区）
    {"scenic_id": "xihu", "kind": "cross_scenic",
     "q": "灵山大佛在你们西湖景区的哪个位置？", "correct_keywords": []},
    {"scenic_id": "lingshan", "kind": "cross_scenic",
     "q": "你们灵山的迎客松在哪条游览路线上？", "correct_keywords": []},
    {"scenic_id": "gugong", "kind": "cross_scenic",
     "q": "故宫里的断桥残雪怎么走？", "correct_keywords": []},
    {"scenic_id": "huangshan", "kind": "cross_scenic",
     "q": "黄山景区的太和殿门票多少钱？", "correct_keywords": []},
]
# ═══════════════════════════════════════════════════════════════════════════
# Tier 4 对抗难题集：口语化 / 别名 / 省略提及冷门景点 —— 不直说景点全名，
# 考验检索层在"游客真实说话方式"下能否召回正确 chunk。
# 用于 EA-RAG vs 基线的检索级 A/B（关键词为知识库原文中的确定性事实）。
# ═══════════════════════════════════════════════════════════════════════════
ADVERSARIAL_CASES = [
    # 灵山（冷门景点 + 口语省略）
    {"scenic_id": "lingshan", "q": "入口那块刻着四个鎏金大字的石壁有多长？",
     "keywords": ["39.8"], "target": "灵山大照壁"},
    {"scenic_id": "lingshan", "q": "那对巨大的青铜佛脚印一只有多长？",
     "keywords": ["1.2"], "target": "佛足坛"},
    {"scenic_id": "lingshan", "q": "五座并排跨在水上的白玉石桥代表什么智慧？",
     "keywords": ["五明", "声明", "因明"], "target": "五明桥"},
    {"scenic_id": "lingshan", "q": "大照壁多长？",
```

```
    "keywords": ["39.8"], "target": "灵山大照壁（省略前缀）"},
  {"scenic_id": "lingshan", "q": "刻着佛陀战胜魔王故事的那面墙讲的是什么？",
    "keywords": ["降魔", "波旬", "成道"], "target": "降魔浮雕"},
  # 拈花湾
  {"scenic_id": "nihuawan", "q": "小镇门口广场上那个微笑主题的雕塑有多高？",
    "keywords": ["12"], "target": "拈花广场"},
  {"scenic_id": "nihuawan", "q": "种了格桑花波斯菊的那片花海有多大？",
    "keywords": ["30000", "3万"], "target": "梵天花海"},
  # 西湖
  {"scenic_id": "xihu", "q": "白娘子和许仙相遇的那座桥在哪里？",
    "keywords": ["白堤", "断桥"], "target": "断桥残雪"},
  {"scenic_id": "xihu", "q": "苏东坡用湖泥堆出来的那条长堤有多长？",
    "keywords": ["2797"], "target": "苏堤春晓"},
  # 黄山
  {"scenic_id": "huangshan", "q": "那棵伸着手臂欢迎客人的千年老松树高多少？",
    "keywords": ["10.6"], "target": "迎客松"},
  {"scenic_id": "huangshan", "q": "黄山上宽只有一米、两边都是深渊的那段路叫什么？",
    "keywords": ["鲫鱼背"], "target": "天都峰鲫鱼背"},
  # 故宫
  {"scenic_id": "gugong", "q": "皇帝上朝那个最大的宫殿面积多少？",
    "keywords": ["2377"], "target": "太和殿"},
  {"scenic_id": "gugong", "q": "藏着皇太子名单的那块匾在哪个宫殿？",
    "keywords": ["乾清宫", "正大光明"], "target": "乾清宫"},
]
def coverage_report() -> dict:
  """统计景点级覆盖率 + 字段维度分布。"""
  recs = load_attractions()
  cov_qs = build_coverage_questions()
  by_scenic = {}
  for r in recs:
    by_scenic.setdefault(r["scenic_id"], set()).add(r["name"])
  asked = {}
  for q in cov_qs:
    asked.setdefault(q["scenic_id"], set()).add(q["attraction"])
  report = {}
  for sid, names in by_scenic.items():
    a = asked.get(sid, set())
    report[sid] = {"total_attractions": len(names),
            "covered": len(a & names),
            "rate": round(len(a & names) / len(names) * 100, 1)}
  total = sum(v["total_attractions"] for v in report.values())
  covered = sum(v["covered"] for v in report.values())
  report["_overall"] = {"total_attractions": total, "covered": covered,
              "rate": round(covered / total * 100, 1)}
  return report
if __name__ == "__main__":
  recs = load_attractions()
  cov = build_coverage_questions()
  cur = build_curated_questions()
  print(f"景点总数: {len(recs)}")
```

```
    print(f"自动覆盖题: {len(cov)}  精选难题: {len(cur)}  陷阱题: {len(TRAP_CASES)}")
    print(f"正常题合计: {len(cov)+len(cur)}")
    print("\n覆盖率报告:")
    for sid, v in coverage_report().items():
        print(f"  {sid}: {v}")
    print("\n样例自动题:")
    for q in cov[:6]:
        print(f"  [{q['scenic_id']}/{q['dimension']}] {q['q']}  KW={q['keywords']}")
==================== tests/eval_run.py ====================
"""
RAG 综合评测运行器 —— 一条命令产出可进 PPT/演示视频的硬证据。
产出指标：
  · 准确率          Σ命中 / 题数          （事实正确性）
  · 有效回答率      正确 ∩ 首句延迟≤阈值    （又对又快，合并准确率+延迟两条红线）
  · 首句/整段延迟   p50 / p95             （数字人「开口时刻」才是感知延迟）
  · 景点覆盖率      被考查景点 / 总景点     （由构造保证 100%，证明面面俱到）
  · 拒答纠错率      陷阱题应对正确 / 陷阱题数 （证明不幻觉、可信赖）
  · 消融对比        有RAG vs 无RAG 准确率   （证明 RAG 真在干活，不是模型本来就会）
运行：
    cd /home/jiale/Desktop/Digital_Person
    .venv/bin/python tests/eval_run.py              # 全量（~105正常题×2 + 15陷阱，约20-30min）
    .venv/bin/python tests/eval_run.py --scenic xihu   # 只测某景区（快速验证）
    .venv/bin/python tests/eval_run.py --limit 8       # 每景区限8题（冒烟）
    .venv/bin/python tests/eval_run.py --no-ablation   # 跳过无RAG对照（省一半时间）
    .venv/bin/python tests/eval_run.py --threshold 3.0 # 有效回答率的首句延迟阈值（秒）
产物：
    tests/eval_report.json   完整逐题明细（供人工复核 / 答辩取证）
    tests/eval_overview.png  一张总览图（直接进 PPT 和演示视频）
"""
import os, sys, json, time, argparse, asyncio, statistics
from pathlib import Path
from dotenv import load_dotenv
ROOT = Path(__file__).resolve().parent.parent
load_dotenv(ROOT / ".env")
for _v in ("ALL_PROXY", "all_proxy", "HTTPS_PROXY", "https_proxy", "HTTP_PROXY", "http_proxy"):
    os.environ.pop(_v, None)
os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
sys.path.insert(0, str(ROOT))
from tests.eval_kb import (
    build_coverage_questions, build_curated_questions, coverage_report,
    TRAP_CASES, REFUSE_MARKERS, SCENIC_ZH,
)
from backend.rag import pipeline
# —— 评测期间禁用答案缓存：每题都要真实冷测（否则消融第二轮会命中第一轮缓存）——
pipeline.answer_cache.get = lambda *a, **k: None
pipeline.answer_cache.set = lambda *a, **k: None
def _hit(reply: str, keywords: list[str]) -> bool:
    """任一关键词命中即算正确（确定性、可肉眼核对）。"""
    return any(kw in reply for kw in keywords) if keywords else False
async def _ask_stream(q: str, scenic_id: str) -> dict:
```

```
    """跑一题流式问答，记录首句延迟 / 整段延迟 / 全文 / 意图。"""
    t0 = time.time()
    first_sentence_t = None
    intent = None
    full = ""
    async for ev in pipeline.answer_stream(q, scenic_id, session_id="eval"):
        if ev["type"] == "intent":
            intent = ev.get("value")
        elif ev["type"] == "sentence" and first_sentence_t is None:
            first_sentence_t = time.time() - t0
        elif ev["type"] == "end":
            full = ev["full_text"]
    total = time.time() - t0
    if first_sentence_t is None:
        first_sentence_t = total
    return {"reply": full, "first_latency": first_sentence_t,
            "total_latency": total, "intent": intent}
# ── 消融：把检索函数替换成返回空，模拟「无 RAG」（裸 LLM 凭参数记忆答）──
_orig_retrieve = pipeline.retrieve
_orig_retrieve_async = getattr(pipeline, "retrieve_async", None)
def _disable_rag():
    pipeline.retrieve = lambda *a, **k: []
    if _orig_retrieve_async:
        async def _empty(*a, **k):
            return []
        pipeline.retrieve_async = _empty
def _enable_rag():
    pipeline.retrieve = _orig_retrieve
    if _orig_retrieve_async:
        pipeline.retrieve_async = _orig_retrieve_async
def run_normal(questions, threshold, label="有RAG"):
    """跑正常题集，返回逐题结果 + 汇总。"""
    rows = []
    print(f"\n{'='*64}\n  {label}  正常题评测（{len(questions)} 题，有效阈值首句≤{threshold}s）\n{'='*64}")
    for i, q in enumerate(questions, 1):
        try:
            r = asyncio.run(_ask_stream(q["q"], q["scenic_id"]))
            hit = _hit(r["reply"], q["keywords"])
            valid = hit and r["first_latency"] <= threshold
            rows.append({**q, **r, "hit": hit, "valid": valid})
            mark = "✓" if hit else "✗"
            vmark = "✗" if valid else " "
            print(f"  [{i:03d}] {mark}{vmark} 首句{r['first_latency']:.1f}s "
                  f"[{q['scenic_id']}/{q['dimension']}] {q['q'][:30]}")
            if not hit:
                print(f"       期望KW={q['keywords']}  答:{r['reply'][:60]}")
        except Exception as e:
            print(f"  [{i:03d}] ✗ ERROR {e}")
            rows.append({**q, "reply": f"ERROR:{e}", "hit": False, "valid": False,
                         "first_latency": 0, "total_latency": 0, "intent": None})
```

```
    return rows
def run_traps(traps):
    rows = []
    print(f"\n{'='*64}\n  陷阱题评测（{len(traps)} 题：拒答 / 纠错能力）\n{'='*64}")
    for i, t in enumerate(traps, 1):
        try:
            r = asyncio.run(_ask_stream(t["q"], t["scenic_id"]))
            reply = r["reply"]
            refused = any(m in reply for m in REFUSE_MARKERS)
            corrected = _hit(reply, t.get("correct_keywords", []))
            ok = refused or corrected
            rows.append({**t, "reply": reply, "refused": refused,
                         "corrected": corrected, "ok": ok,
                         "first_latency": r["first_latency"]})
            print(f"  [{i:02d}] {'✓' if ok else '✗'} [{t['kind']}] {t['q'][:32]}")
            if not ok:
                print(f"       答:{reply[:80]}")
        except Exception as e:
            print(f"  [{i:02d}] ✗ ERROR {e}")
            rows.append({**t, "reply": f"ERROR:{e}", "ok": False})
    return rows
def summarize(rows):
    n = len(rows)
    if n == 0:
        return {}
    hits = sum(r["hit"] for r in rows)
    valids = sum(r["valid"] for r in rows)
    firsts = [r["first_latency"] for r in rows if r.get("first_latency")]
    totals = [r["total_latency"] for r in rows if r.get("total_latency")]
    def pct(x): return round(x / n * 100, 1)
    def p(vals, q):
        if not vals: return 0.0
        vals = sorted(vals)
        k = max(0, min(len(vals) - 1, int(round(q * (len(vals) - 1)))))
        return round(vals[k], 2)
    # 各景区准确率
    by_scenic = {}
    for r in rows:
        d = by_scenic.setdefault(r["scenic_id"], [0, 0])
        d[0] += int(r["hit"]); d[1] += 1
    scenic_acc = {s: round(h / t * 100, 1) for s, (h, t) in by_scenic.items()}
    return {
        "n": n, "accuracy": pct(hits), "valid_rate": pct(valids),
        "first_p50": p(firsts, .5), "first_p95": p(firsts, .95),
        "total_p50": p(totals, .5), "total_p95": p(totals, .95),
        "scenic_accuracy": scenic_acc,
    }
def make_chart(report, out_png, threshold):
    import matplotlib
    matplotlib.use("Agg")
```

```
import matplotlib.pyplot as plt
from matplotlib import font_manager
# 直接按文件路径注册 CJK 字体，避免「按family名找不到」导致中文显示成方框
for fp in ("/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc",
           "/usr/share/fonts/truetype/wqy/wqy-zenhei.ttc"):
    if Path(fp).exists():
        font_manager.fontManager.addfont(fp)
        plt.rcParams["font.sans-serif"] = [font_manager.FontProperties(fname=fp).get_name()]
        break
plt.rcParams["axes.unicode_minus"] = False
main = report["rag"]; ablt = report.get("ablation"); trap = report["trap"]
cov = report["coverage"]["_overall"]["rate"]
fig, axes = plt.subplots(2, 2, figsize=(13, 9))
fig.suptitle("景区导览 RAG 数字人 · 问答质量综合评测", fontsize=16, fontweight="bold")
# 1) 核心指标
ax = axes[0, 0]
labels = ["准确率", f"有效回答率\n(正确∩首句≤{threshold}s)", "拒答纠错率", "景点覆盖率"]
vals = [main["accuracy"], main["valid_rate"], trap["rate"], cov]
bars = ax.bar(labels, vals, color=["#2E86DE", "#27AE60", "#E67E22", "#8E44AD"])
ax.axhline(90, ls="--", c="red", lw=1.2); ax.text(3.4, 91, "90%红线", color="red", fontsize=9)
ax.set_ylim(0, 105); ax.set_ylabel("%"); ax.set_title("核心指标")
for b, v in zip(bars, vals):
    ax.text(b.get_x()+b.get_width()/2, v+1, f"{v}%", ha="center", fontsize=10, fontweight="bold")
# 2) 消融对比
ax = axes[0, 1]
if ablt:
    bars = ax.bar(["无 RAG\n(裸LLM)", "有 RAG\n(本系统)"], [ablt["accuracy"], main["accuracy"]],
                  color=["#BDC3C7", "#2E86DE"])
    ax.set_ylim(0, 105); ax.set_ylabel("准确率 %"); ax.set_title("消融实验：RAG 的贡献")
    for b, v in zip(bars, [ablt["accuracy"], main["accuracy"]]):
        ax.text(b.get_x()+b.get_width()/2, v+1, f"{v}%", ha="center", fontsize=11, fontweight="bold")
    gain = round(main["accuracy"] - ablt["accuracy"], 1)
    ax.text(0.5, 50, f"+{gain}%\nRAG净增益", ha="center", color="#C0392B", fontsize=13, fontweight="bold")
else:
    ax.text(0.5, 0.5, "（已跳过消融对比）", ha="center", va="center"); ax.axis("off")
# 3) 延迟
ax = axes[1, 0]
labels = ["首句 p50", "首句 p95", "整段 p50", "整段 p95"]
vals = [main["first_p50"], main["first_p95"], main["total_p50"], main["total_p95"]]
bars = ax.bar(labels, vals, color=["#27AE60", "#52BE80", "#5DADE2", "#85C1E9"])
ax.axhline(5, ls="--", c="red", lw=1.2); ax.text(3.3, 5.1, "5s红线", color="red", fontsize=9)
ax.axhline(threshold, ls=":", c="#E67E22", lw=1.2)
ax.set_ylabel("秒"); ax.set_title("响应延迟（首句=数字人开口时刻）")
for b, v in zip(bars, vals):
    ax.text(b.get_x()+b.get_width()/2, v+0.05, f"{v}s", ha="center", fontsize=10, fontweight="bold")
# 4) 各景区准确率
ax = axes[1, 1]
sa = main["scenic_accuracy"]
names = [SCENIC_ZH.get(s, s) for s in sa]
vals = list(sa.values())
```

```
    bars = ax.bar(names, vals, color="#16A085")
    ax.axhline(90, ls="--", c="red", lw=1.2)
    ax.set_ylim(0, 105); ax.set_ylabel("准确率 %"); ax.set_title("各景区准确率（插拔式知识库泛化）")
    for b, v in zip(bars, vals):
        ax.text(b.get_x()+b.get_width()/2, v+1, f"{v}%", ha="center", fontsize=9, fontweight="bold")
    fig.tight_layout(rect=[0, 0, 1, 0.96])
    fig.savefig(out_png, dpi=140)
    print(f"\n总览图已保存: {out_png}")
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--scenic", nargs="*", help="只测指定景区")
    ap.add_argument("--limit", type=int, default=0, help="每景区最多题数（冒烟用）")
    ap.add_argument("--threshold", type=float, default=3.0, help="有效回答率的首句延迟阈值(秒)")
    ap.add_argument("--no-ablation", action="store_true", help="跳过无RAG对照")
    ap.add_argument("--no-trap", action="store_true", help="跳过陷阱题")
    args = ap.parse_args()
    questions = build_coverage_questions() + build_curated_questions()
    traps = TRAP_CASES
    if args.scenic:
        questions = [q for q in questions if q["scenic_id"] in args.scenic]
        traps = [t for t in traps if t["scenic_id"] in args.scenic]
    if args.limit:
        per = {}
        kept = []
        for q in questions:
            c = per.get(q["scenic_id"], 0)
            if c < args.limit:
                kept.append(q); per[q["scenic_id"]] = c + 1
        questions = kept
    # 预热：首次调用要加载 BGE-M3 + Reranker 模型（冷启动 ~10s），生产环境服务常驻是热的，
    # 这一刀不计入延迟统计，否则 p95 被冷启动污染、不代表真实体验。
    if questions:
        print("预热中（加载嵌入/重排模型，不计时）...")
        try:
            asyncio.run(_ask_stream(questions[0]["q"], questions[0]["scenic_id"]))
        except Exception as e:
            print(f"  预热异常（忽略）: {e}")
    t_start = time.time()
    rag_rows = run_normal(questions, args.threshold, "有RAG")
    rag_sum = summarize(rag_rows)
    abl_sum = None
    abl_rows = []
    if not args.no_ablation:
        _disable_rag()
        abl_rows = run_normal(questions, args.threshold, "无RAG(消融)")
        abl_sum = summarize(abl_rows)
        _enable_rag()
    trap_rows = []
    trap_sum = {"rate": 0.0, "n": 0}
    if not args.no_trap:
```

```
    trap_rows = run_traps(traps)
    ok = sum(r["ok"] for r in trap_rows)
    trap_sum = {"rate": round(ok / len(trap_rows) * 100, 1) if trap_rows else 0.0,
                "n": len(trap_rows), "ok": ok}
  report = {
    "generated_at": time.strftime("%Y-%m-%d %H:%M:%S"),
    "elapsed_sec": round(time.time() - t_start, 1),
    "threshold_s": args.threshold,
    "coverage": coverage_report(),
    "rag": rag_sum, "ablation": abl_sum, "trap": trap_sum,
    "detail": {"rag": rag_rows, "ablation": abl_rows, "trap": trap_rows},
  }
  # ── 终端汇总 ──
  print(f"\n{'#'*64}\n  综合评测结果\n{'#'*64}")
  print(f"  景点覆盖率  : {report['coverage']['_overall']['rate']}%  "
      f"({report['coverage']['_overall']['covered']}/{report['coverage']['_overall']['total_attractions']}
   景点)")
  print(f"  准确率      : {rag_sum['accuracy']}%  ({rag_sum['n']} 题)")
  print(f"  有效回答率  : {rag_sum['valid_rate']}%  (正确且首句≤{args.threshold}s)")
  print(f"  首句延迟    : p50={rag_sum['first_p50']}s  p95={rag_sum['first_p95']}s")
  print(f"  整段延迟    : p50={rag_sum['total_p50']}s  p95={rag_sum['total_p95']}s")
  if abl_sum:
    print(f"  消融对比    : 无RAG={abl_sum['accuracy']}% → 有RAG={rag_sum['accuracy']}%  "
        f"(+{round(rag_sum['accuracy']-abl_sum['accuracy'],1)}%)")
  if trap_rows:
    print(f"  拒答纠错率  : {trap_sum['rate']}%  ({trap_sum['ok']}/{trap_sum['n']} 陷阱题)")
  print(f"  各景区准确率 : {rag_sum['scenic_accuracy']}")
  print(f"  总耗时      : {report['elapsed_sec']}s")
  out_json = ROOT / "tests" / "eval_report.json"
  out_json.write_text(json.dumps(report, ensure_ascii=False, indent=2), encoding="utf-8")
  print(f"\n详细报告已保存: {out_json}")
  make_chart(report, ROOT / "tests" / "eval_overview.png", args.threshold)
if __name__ == "__main__":
  main()
==================== tests/eval_ea_rag.py ====================
"""
EA-RAG 消融实验：检索级 A/B 对比（不调 LLM，快、便宜、指标纯净）。
为什么测"检索级"而非端到端：EA-RAG 改的是检索编排层，直接度量
「正确事实是否被召回进 top-k chunk」最能隔离算法本身的贡献——
召回对了，LLM 几乎必答对（端到端105题已另行验证）。
指标：
 · 召回命中率 Recall@k —— 任一期望关键词出现在返回的 top-k chunk 中
 · 误拒率 False-Refusal —— 正常题被门控误判为"无信息"的比例（应≈0）
 · 拒答率 Refusal@OOK  —— 知识库外陷阱题被正确判空的比例（越高越好）
消融配置（逐机制叠加）：
 baseline  全关（原始拼接去重行为）
 +anchor   仅实体锚定
 +fusion   仅自适应融合
 EA-RAG    全开（含门控）
运行：
```

```
  .venv/bin/python tests/eval_ea_rag.py          # 全量消融
  .venv/bin/python tests/eval_ea_rag.py --quick   # 只跑对抗集（快速验证）
"""
import os, sys, json, time, argparse
from pathlib import Path
from dotenv import load_dotenv
ROOT = Path(__file__).resolve().parent.parent
load_dotenv(ROOT / ".env")
for _v in ("ALL_PROXY", "all_proxy", "HTTPS_PROXY", "https_proxy", "HTTP_PROXY", "http_proxy"):
  os.environ.pop(_v, None)
os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
sys.path.insert(0, str(ROOT))
from tests.eval_kb import (
  build_coverage_questions, build_curated_questions,
  ADVERSARIAL_CASES, TRAP_CASES,
)
from backend.core.config import settings
from backend.rag.retriever import retrieve
CONFIGS = [
  # (名称, entity_anchor, adaptive_fusion, confidence_gate)
  ("baseline", False, False, False),
  ("+anchor",  True,  False, False),
  ("+fusion",  False, True,  False),
  ("EA-RAG",   True,  True,  True),
]
def _apply(anchor: bool, fusion: bool, gate: bool):
  settings.ea_entity_anchor = anchor
  settings.ea_adaptive_fusion = fusion
  settings.ea_confidence_gate = gate
def _recall_hit(chunks: list[str], keywords: list[str]) -> bool:
  """任一关键词出现在任一召回 chunk 中即算召回命中。"""
  text = "\n".join(chunks)
  return any(kw in text for kw in keywords)
def run_config(name: str, normal: list[dict], adv: list[dict], ook: list[dict]) -> dict:
  """跑一个配置：正常题 + 对抗题召回率，正常题误拒率，知识库外拒答率。"""
  n_hit = n_refused = 0
  for q in normal:
    chunks = retrieve(q["q"], q["scenic_id"])
    if not chunks:
      n_refused += 1
    elif _recall_hit(chunks, q["keywords"]):
      n_hit += 1
  a_hit = 0
  adv_miss = []
  for q in adv:
    chunks = retrieve(q["q"], q["scenic_id"])
    if chunks and _recall_hit(chunks, q["keywords"]):
      a_hit += 1
    else:
      adv_miss.append(q["target"])
```

```
    o_refused = 0
    for t in ook:
        chunks = retrieve(t["q"], t["scenic_id"])
        if not chunks:
            o_refused += 1
    r = {
        "normal_recall": round(n_hit / len(normal) * 100, 1) if normal else 0,
        "false_refusal": round(n_refused / len(normal) * 100, 1) if normal else 0,
        "adv_recall": round(a_hit / len(adv) * 100, 1) if adv else 0,
        "ook_refusal": round(o_refused / len(ook) * 100, 1) if ook else 0,
        "adv_miss": adv_miss,
    }
    print(f"  {name:<10} 正常召回 {r['normal_recall']:>5}%   对抗召回 {r['adv_recall']:>5}%   "
          f"误拒 {r['false_refusal']:>4}%   库外拒答 {r['ook_refusal']:>5}%")
    if adv_miss:
        print(f"  {'':<10} 对抗漏检: {adv_miss}")
    return r
def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--quick", action="store_true", help="只跑对抗集")
    args = ap.parse_args()
    normal = [] if args.quick else build_coverage_questions() + build_curated_questions()
    adv = ADVERSARIAL_CASES
    ook = [t for t in TRAP_CASES if t["kind"] == "out_of_kb"]
    print(f"题量：正常 {len(normal)} / 对抗 {len(adv)} / 知识库外 {len(ook)}")
    print("预热（加载嵌入与重排模型）...")
    _apply(False, False, False)
    retrieve("预热", "lingshan")
    results = {}
    t0 = time.time()
    print(f"\n{'='*84}")
    for name, anchor, fusion, gate in CONFIGS:
        _apply(anchor, fusion, gate)
        results[name] = run_config(name, normal, adv, ook)
    print(f"{'='*84}")
    print(f"总耗时 {time.time()-t0:.0f}s")
    # 恢复默认全开
    _apply(True, True, True)
    out = ROOT / "tests" / "ea_rag_ablation.json"
    out.write_text(json.dumps({
        "generated_at": time.strftime("%Y-%m-%d %H:%M:%S"),
        "n_normal": len(normal), "n_adv": len(adv), "n_ook": len(ook),
        "results": results,
    }, ensure_ascii=False, indent=2), encoding="utf-8")
    print(f"报告已保存: {out}")
if __name__ == "__main__":
    main()
```